GA-DRL: Graph Neural Network-Augmented Deep Reinforcement Learning for DAG Task Scheduling Over Dynamic Vehicular Clouds

Zhang Liu, *Graduate Student Member, IEEE*, Lianfen Huang, *Member, IEEE*, Zhibin Gao, *Member, IEEE*, Manman Luo, Seyyedali Hosseinalipour, *Member, IEEE*, and Huaiyu Dai, *Fellow, IEEE*

***Abstract*—Vehicular Clouds (VCs) are modern platforms for processing of computation-intensive tasks over vehicles. Such tasks are often represented as Directed Acyclic Graphs (DAGs) consisting of interdependent vertices/subtasks and directed edges. However, efficient scheduling of DAG tasks over VCs presents significant challenges, mainly due to the dynamic service provisioning of vehicles within VCs and non-Euclidean representation of DAG tasks' topologies. In this paper, we propose a Graph neural network-Augmented Deep Reinforcement Learning scheme (GA-DRL) for the timely scheduling of DAG tasks over dynamic VCs. In doing so, we first model the VC-assisted DAG task scheduling as a Markov decision process. We then adopt a multi-head Graph ATtention network (GAT) to extract the features of DAG subtasks. Our developed GAT enables a two-way aggregation of the topological information in a DAG task by simultaneously considering predecessors and successors of each subtask. We further introduce non-uniform DAG neighborhood sampling through codifying the scheduling priority of different subtasks, which makes our developed GAT generalizable to completely unseen DAG task topologies. Finally, we augment GAT into a double deep Q-network learning module to conduct subtask-to-vehicle assignment according to the extracted features of subtasks, while considering the dynamics and heterogeneity of the vehicles in VCs. Through simulating various DAG tasks under real-world movement traces of vehicles, we demonstrate that GA-DRL outperforms existing benchmarks in terms of DAG task completion time.**

***Index Terms*—Vehicular cloud, DAG scheduling, deep reinforcement learning, graph neural network.**

Manuscript received 30 August 2023; revised 27 December 2023 and 2 April 2024; accepted 3 April 2024. Date of publication 11 April 2024; date of current version 21 August 2024. The work presented in this paper was partially supported by the National Natural Science Foundation of China (grant nos. 62371406, 61971365, 62171392); the Natural Science Foundation of Fujian Province of China (grant nos. 2020H6001, 2021H6001, 2021J01004), and the Xiamen Major science and Technology Project (grant nos. 3502Z20221026). The associate editor coordinating the review of this article and approving it for publication was O. J. Pandey. *(Corresponding author: Lianfen Huang.)*

Zhang Liu and Lianfen Huang are with the Department of Informatics and Communication Engineering, Xiamen University, Xiamen 361005, China (e-mail: zhangliu@stu.xmu.edu.cn; lfhuang@xmu.edu.cn).

Zhibin Gao is with the Navigation Institute, Jimei University, Xiamen 361021, China (e-mail: gaozhibin@jmu.edu.cn).

Manman Luo is with the Department of Electronic Engineering, Xiamen University, Xiamen 361005, China (e-mail: luomanman@stu.xmu.edu.cn).

Seyyedali Hosseinalipour is with the Department of Electrical Engineering, University at Buffalo SUNY, Buffalo, NY 14260 USA (e-mail: alipour@buffalo.edu).

Huaiyu Dai is with the Department of Electrical and Computer Engineering, North Carolina State University, Raleigh, NC 26795 USA (e-mail: hdai@ncsu.edu).

Digital Object Identifier 10.1109/TNSM.2024.3387707

I. Introduction

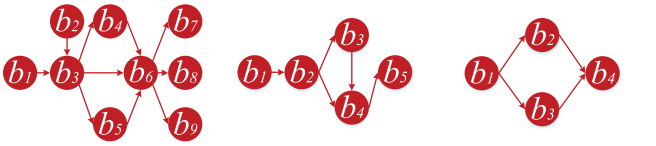
A. Background and Challenges

With the rapid development of connected devices in Internet of Things (IoT) ecosystem, especially smart cars, vehicular networks have been envisioned to provide a reliable platform for execution of a myriad of vehicular applications/tasks, such as on-board Augmented Reality/Virtual Reality (AR/VR) and face recognition [1], [2]. Many of these tasks are computation-intensive and possess complex computation topologies, which are represented as Directed Acyclic Graphs (DAGs) [28]. Fig. 1 illustrates some examples of real-world mobile DAG applications/tasks and their inherent interdependencies among different subtasks [27], [53]. Take navigator task as an example, where vertices denote subtasks of the task and directed edges describe the dependencies between the execution of subtasks. In particular, each subtask represents a *processing component* of navigation, while directed edges dictate the *sequence of executions* of subtasks. The sequential execution of subtasks in a DAG model stems from the fact that processing of a subtask may depend on the output data of others (e.g., for navigator task, processing of subtask b_4 relies on the output data of subtask b_3 and processing of b_6 relies on the output data of both subtasks b_3, b_4 and b_5).

In vehicular networks, DAG tasks are frequently encountered. Nevertheless, one of the main obstacles in the execution of DAG tasks is that a task owner (i.e., a vehicle with a DAG task) in a vehicular network often fails to fulfill the task's execution requirements due to its limited on-board computation resources. To circumvent this, offloading the computation of DAG tasks from task owners to edge servers through the Mobile Edge Computing (MEC) platform has been proposed [12], [13], [14]. However, such task offloading strategies often rely on Vehicle-to-Infrastructure (V2I) communications, which can suffer from a high latency (e.g., due to a high data traffic congestion on the fronthaul/backhaul links) and limited coverage (e.g., in suburban areas) [9]. In response to these limitations, Vehicular Clouds (VCs) have emerged as novel computing platforms that integrate heterogeneous and distributed computation resources of moving vehicles

1932-4537 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.



(a) Navigator application. (b) Public face recognition application. (c) Target tracking application.

Fig. 1. DAG models of real-world mobile applications with their inherent structures [27], [53].

via opportunistic Vehicle-to-Vehicle (V2V) communications to build flexible and scalable computing topologies for real-time task processing [15], [16], [17]. Specifically, in a VC, DAG subtasks are dispersed across vehicles and the data needed for the execution of subtasks is transmitted via V2V links.

Although DAG task processing over VCs is promising, efficient scheduling of DAG subtasks across vehicles is a highly non-trivial problem, which resembles Mixed Integer Programming (MIP) due to the existence of continuous and binary variables in the formulation (detailed in Section III-E). MIP are NP-hard problems [43], for which, dynamic programming [4], [7] and list scheduling algorithms [5], [12] have been widely used to obtain solutions. However, considering the unique characteristics of VCs, especially the high mobility of vehicles, varying V2V channel conditions and intermittent V2V connectivity, the processing of DAG tasks may face disruptions once their subtasks are dispersed over dynamic vehicles in a VC. This disruption necessitates a carefully crafted subtask-to-vehicle allocation algorithm that is capable of dispersing DAG substasks across vehicles in a timely manner, while taking into account the unique processing topology of DAG tasks and the dynamics of vehicles and their resources in a VC. Nevertheless, the aforementioned algorithms often suffer from a prohibitively high computation complexity, which renders them impractical for large-scale VCs. Also, these algorithms require a prior knowledge about the system dynamics (e.g., time-varying V2V channel qualities), which is cumbersome to acquire in practical systems. To overcome these challenges, researchers have recently started exploring the learning-based methods, a popular example of which is Deep Reinforcement Learning (DRL) [20], [21], [22], [23], [24], [25], [26]. Roughly speaking, DRL learns from interacting with an environment so as to generate real-time near-optimal mappings from the state space (detailed in Section IV-C) to the action space (detailed in Section IV-D) without requiring any prior knowledges about the environment.

Although DRL has shown a tremendous success in task scheduling/offloading [20], [21], [22], [23], [24], [25], [26], it cannot be readily adopted for scheduling of DAG tasks over dynamic VCs. This is due to the fact that DAG tasks' data (i.e., tasks' topologies) resides in a non-Euclidean space (i.e., graph). As a result, conventional DRL with handcrafted states designed to work with data in Euclidean spaces fails to automatically learn the topological information of DAG tasks, and thus can be hardly applied to unseen DAG task topologies upon deployment in real-world systems. To overcome this challenge, we propose to augment DRL with an emerging learning architecture called Graph Neural Network (GNN). GNNs are capable of adaptively extracting discriminative features for each node of a graph based on the topological information aggregated from its neighboring nodes [41]. As a sub-category of GNNs, Graph ATtention networks (GATs) have recently gained tremendous attentions, which extend the spatial convolution in Convolutional Neural Networks (CNNs) to graph structures [42] and benefit from *inductive learning*, making their learned models generalizable to unseen graph topologies.

B. Overview and Summary of Contributions

Inspired by the unique advantages of DRL and GNNs, we propose GA-DRL, a GNN-augmented DRL scheme to conduct DAG subtask-to-vehicle allocation aiming at minimizing the DAG task completion time. In doing so, we first model the VC-assisted DAG task scheduling as a Markov Decision Process (MDP). We then tailor a GAT to extract a set of features for each subtask of a DAG task. Finally, we integrate our developed GAT into the learning architecture of a Double Deep Q-Network (DDQN) to generate subtask-to-vehicle allocation decisions, while taking into account the dynamics and heterogeneity of vehicles in a VC. Major contributions of this paper can be summarized as follows:

- We develop a multi-head GAT capable of extracting features for DAG subtasks. Particularly, our GAT conducts a two-way topological information aggregation by simultaneously considering predecessors and successors of each subtask. Further, we incorporate a non-uniform neighborhood sampling methodology into our GAT by codifying the scheduling priority of subtasks, making our GAT generalizable to unseen DAG task topologies upon being deployed over the real-world systems.
- We propose a DDQN to conduct subtask-to-vehicle allocation decisions according to the extracted features of subtasks by our GAT, while taking into account the dynamics and heterogeneity of vehicles in a VC. Further, we incorporate an action mask module into the DDQN to avoid infeasible subtask-to-vehicle allocations, ensuring successful execution of subtasks.
- We evaluate the performance of GA-DRL on a real-world road network obtained from OpenStreetMap [10] where SUMO [11], one of the most popular softwares for generating traffic flow, is used to form a VC. Through simulating DAG tasks with various topologies, we reveal that GA-DRL can outperform existing benchmarks in terms of task completion time.

The rest of this paper is organized as follows: Section II contains the related work. In Section III, we present the system model and formulate the VC-assisted DAG task scheduling as a MIP problem. We develop GA-DRL in Section IV. In Section V, we present simulation results before concluding the paper in Section VI.

II. Related Work

Existing works on DAG task scheduling over cloud-assisted networks can be roughly divided into two categories with respect to the type of their scheduling mechanisms:

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.

i) heuristic-based algorithms [3], [4], [6], [12], [13], [14], [15], [17]; *ii*) learning-based methods [16], [18], [20], [21], [22], [23], [24], [25], [26]. Below, we summarize the contributions of these works, and highlight the differences between our methodology in this paper and prior works.

A. Heuristic DAG Task Scheduling

1) Static Computing Environment: Heuristic methods for DAG task scheduling have been extensively studied for static MEC networks with fully connected servers [3], [4], [6], [12], [13]. Topcuoglu et al. in [12] proposed HEFT algorithm, where each subtask is assigned to the processor with the least execution time. In [3], Bittencourt et al. proposed forward looking attributions to improve the performance of HEFT. In [6], Kanemitsu et al. proposed a clustering-based DAG task scheduling algorithm via prioritizing assigning the subtasks located on the critical path to the same processor. Sih et al. in [4] adopted a compile-time-aware scheduling algorithm to dynamically allocate DAG subtasks over the existing processing units in the system. Recently, in [13], Sahni et al. introduced JDOFH to simultaneously consider dependencies among DAG subtasks and start time of network flows to transmit the data of subtasks over the network.

2) Dynamic Computing Environment: Few recent works have studied DAG task scheduling over dynamic networks [14], [15], [17]. Shen et al. in [14] proposed DTOSC to conduct DAG task offloading and service caching in vehicular edge computing. Sun et al. in [15] addressed DAG task scheduling over VC via a modified genetic algorithm focusing on vehicles' dwell times. In [17], Liu et al. developed MAMTS to prioritize allocation of different DAG tasks according to their computation topologies in vehicular edge computing.

The methodologies developed in the aforementioned works are heuristic, applying of which requires considerable number of iterations to reach locally optimal solutions. As a result they often suffer from prohibitively high computation complexities, which renders them impractical for real-time DAG task allocation. Also, these heuristic algorithms often presume a prior knowledge about the system dynamics (e.g., known time-varying V2V channel qualities), obtaining of which is extremely challenging in dynamic VCs, where the network topology may exhibit a significant temporal variation.

B. Learning-Based DAG Task Scheduling

1) Static Computing Environment: DRL schemes have become one of the most popular learning-based techniques in the literature of task scheduling, especially for static MEC networks [20], [21], [22], [23], [24]. In [20], Yan et al. proposed an actor-critic DRL to learn the optimal DAG subtask assignment to access points. Mekala et al. in [21] developed a DRL-based DAG task offloading approach to reduce the utilization cost of edge servers. In [22], Wang et al. proposed a DAG task offloading methodology based on meta reinforcement learning. Goudarzi et al. in [23] introduced weighted actor-learner architectures for DAG task allocation over resource-constrained IoT devices. In [24], Hu et al. presented a DRL-based Monte-Carlo tree search method to minimize DAG tasks' completion times through a clustered scheduler.

2) Dynamic Computing Environment: Considering dynamic computing environments [16], [18], [25], [26], Liu et al. in [16] utilized a policy-based DRL for minimizing DAG tasks' completion times in multi-vehicle scenarios. In [18], Shi et al. proposed a DRL-based DAG task offloading scheme for vehicular fog computing considering the vehicles' mobility and availability. Wei et al. in [25] developed a DRL-based algorithm to jointly optimize the unmanned aerial vehicle trajectory planning, and DAG task scheduling. In [26], Geng et al. proposed a multi-agent actor-critic DRL to schedule DAG tasks in a vehicular edge computing network.

The DRL algorithms developed in the above works are based on handcrafted features, making them unable to fully capture the existing topological information in DAG tasks. This is because the state space of the DRL architectures studied in the above works merely contains basic, human-selected information regarding subtasks (e.g., their computation workloads, transmission data sizes, and number of predecessors/successors). As a result, the DRL methods explored in the above works are solely capable of making allocation decisions for DAG tasks with computation topologies that they have seen during their training period. In this work, we take the first steps towards addressing this limitation.

C. Footprints of GNNs in Mobile Edge Computing

Recently, the success of GNNs in solving a variety of complex problems in wireless communications has been revealed [34], [35], [36], [37], while studying their application in the context of DAG task scheduling is still in early stages. In [34], He et al. investigated the spectrum allocation in vehicle-to-everything networks based on the integration of GNNs and deep Q-learning. Li et al. in [35] proposed a meta-reinforcement learning method for DAG task offloading in MEC platform, where the interdependencies between subtasks was extracted by GNNs. In [36], Lee et al. developed a Graph Convolution Network (GCN) and DRL to effectively learn a priority-based scheduling policy for DAG tasks. Chen et al. in [37] proposed an algorithm called ACED for DAG task offloading, where a GCN is leveraged to capture the topological information of DAG subtasks.

The aforementioned works either ignore the topology of computation-intensive tasks (e.g., interdependencies among subtasks) [34] or focus on static MEC environments, overlooking the dynamics and instability of resource provisioning [35], [36], [37], which are significant features of VCs. Moreover, the GCN architecture developed in [36], [37] relies on *transductive learning*, which requires knowing the graph structure of DAG tasks upfront. As a result, their learned solutions for DAG task scheduling are not applicable to unseen DAG task topologies, which makes them suffer from a prohibitively high training overhead for each newly arrived DAG task to the system. In this work, we are particularly interested to address the shortcomings mentioned above.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SUMMARY OF THE NOTATIONS USED IN OUR PROBLEM FORMULATION

$\mathcal{V}$	Set of vehicles covered by RSU
$\mathcal{B}$	Set of subtasks contained by DAG
$\mathcal{E}$	Set of edges associated with subtasks in $\mathcal{B}$
$\mathcal{P}_i$	Set of immediate predecessors of subtask b_i
$\mathcal{S}_i$	Set of immediate successors of subtask b_i
$\mathcal{B}_m$	Set of subtasks executed on vehicle v_m
f_m	Computation capability of vehicle v_m
$d_{m,n}(t)$	Euclidean distance between vehicles v_m and v_n at slot t
u_i	Computation workload of subtask b_i
$c_{i,j}$	Data transmission size between subtasks b_i and b_j
AT_m	Arrival time of vehicle v_m at VC
DT_m	Departure time of vehicle v_m from VC
$TT_{i,m;j,n}$	Data transmission time between subtasks b_i and b_j when they are processed on vehicle v_m, v_n, respectively
$EST_{i,m}$	Earliest start time of processing subtask b_i on vehicle v_m
$AVT_{i,m}$	Available time of vehicle v_m when it is ready to process subtask b_i
AFT_i	Actual finish time of subtask b_i when it is practically processed via a specific vehicle
$EFT_{i,m}$	Earliest finish time of processing subtask b_i on v_m
RT_i	Ready time of subtask b_i when all of its predecessors have been completed
$\xi_{i,m}$	Binary variable indicating whether subtask b_i is allocated to vehicle v_m

III. System Model and Problem Formulation

In this section, we first give an overview of the system of our interest, DAG task model, vehicle mobility model, and computation offloading model. We then obtain an optimization formulation for VC-assisted DAG task scheduling. Table I summarizes the major notations used in this section.

A. System Overview

We consider a time-slotted VC-assisted DAG task scheduling scenario, which is coordinated by a Road Side Unit (RSU) with coverage diameter of D. We presume that the area comprises $|\mathcal{V}|$ vehicles collected by the set $\mathcal{V} = \{v_m \mid 1 \leq m \leq |\mathcal{V}|\}$. In order to fulfill its DAG task completion demands, a task owner engages in offloading its DAG task with $|\mathcal{B}|$ subtasks collected by the set $\mathcal{B} = \{b_i \mid 1 \leq i \leq |\mathcal{B}|\}$ to other vehicles.[1]

Fig. 2 shows a schematic of VC-assisted cooperative DAG task scheduling scenario of our interest, where target tracking application given in Fig. 1 is allocated among different vehicles and subtask b_0 is a virtual subtask executed on the task owner (detailed in Section III-B). After receiving the offloading request from the task owner (i.e., v_1), the RSU acts as a centralized coordinator [16], which processes a set of collected data (e.g., locations and resources of vehicles) to assign DAG subtasks to vehicles. Specifically, in Fig. 2, virtual subtask b_0 is assumed to be executed on the task owner locally, while subtask b_1 is allocated to vehicle v_3. After executing subtask b_1, vehicle v_3 is scheduled to transmit the output data of subtask b_1 to vehicles v_2 and v_4 for processing subtasks b_2 and b_3. Due to the interdependencies among DAG subtasks, the execution of subtask b_4 relies on the output data of both subtasks b_2 and b_3. Hence, vehicles v_2 and v_4 will both be scheduled to transmit their output data to vehicle v_5. Finally, vehicle v_5 will send a feedback (i.e., the final result of DAG task processing) to the RSU. The main assumptions made in this paper are summarized below:

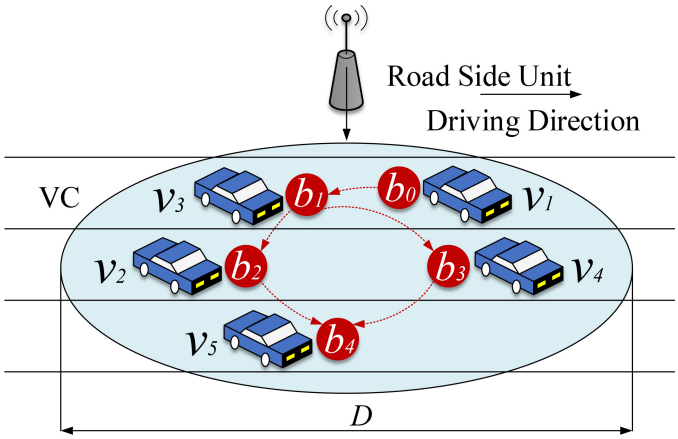


Fig. 2. VC-assisted cooperative DAG task scheduling scenario.

- It is assumed that VC remains stationary during each time slot [39].
- We presume that a single vehicle can only handle one subtask at a time [21]. Consequently, if multiple subtasks are assigned to a vehicle, they must wait until resources become available.[2]
- Due to the mobility and the limited contact durations among vehicles, this paper only focuses on one-hop data transmission between vehicles [38].
- Since the size of the feedback sent to the RSU is usually smaller than that of the original input data, the time it takes to transmit this feedback is neglected [30].

B. DAG Task Model

Without loss of generality, we index the task owner as v_1 with a computation-intensive DAG task, which is represented by a graph $\mathcal{G} = (\mathcal{B}, \mathcal{E})$. In graph $\mathcal{G}$, $\mathcal{B} = \{b_i \mid 1 \leq i \leq |\mathcal{B}|\}$ denotes the set of subtasks, and $\mathcal{E}$ denotes the set of directed edges, where $e_{i,j} \in \mathcal{E}$ indicates that subtask b_i has to be completed before the execution of subtask b_j. To better capture the sequential execution nature of DAG tasks, we further define the set of immediate *predecessors* of each

[1]This paper investigates the DAG task scheduling problem for a single task owner with a single DAG task in one VC for analytical simplicity. Cooperations and resource sharing among VCs and competitions between multiple task owners to acquire computation resources are left as future work.

[2]Upon having vehicles that can process multiple subtasks simultaneously, those vehicles can be modeled as multiple virtual vehicles with unlimited contact duration among them, where each of them can process one subtask at a time.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.

subtask b_i as $\mathcal{P}_i = \{b_j \mid b_j \in \mathcal{B}, e_{j,i} \in \mathcal{E}\}$. Similarly, we define the set of immediate *successors* of each subtask b_i as $\mathcal{S}_i$. For example, in Fig. 1, we have $\mathcal{B} = \{b_1, b_2, b_3, b_4\}$, $\mathcal{E} = \{e_{1,2}, e_{1,3}, e_{2,4}, e_{3,4}\}$, $\mathcal{P}_4 = \{b_2, b_3\}$, and $\mathcal{S}_1 = \{b_2, b_3\}$. Furthermore, to make our analysis tractable, we introduce a virtual subtask to the DAG task topology denoted by b_0, which is connected to subtask(s) with no immediate predecessors as shown in Fig. 2.

C. Vehicle Mobility Model

We assume that each vehicle v_m is driving at a random and constant speed g_m (meters per second). Since the speeds of vehicles are non-negative, we adopt a truncated Gaussian distribution [32] to capture them. Specifically, for any value of speed g, the probability density function of the truncated Gaussian distribution is defined as

$$\widehat{F}(g) = \frac{2F(g)}{\Phi\left(\frac{g_{\max}-\mu_g}{\sigma_g\sqrt{2}}\right) - \Phi\left(\frac{g_{\min}-\mu_g}{\sigma_g\sqrt{2}}\right)}, \tag{1}$$

where $\Phi(x) = \frac{2}{\sqrt{2\pi}}\int_0^x e^{-t^2}dt$ is the Gaussian error function, and $g_{\max}$ and $g_{\min}$ are defined as the maximum and minimum speed of vehicles, respectively. In (1), $F(g)$ is the probability density function of a Gaussian distribution which is given by

$$F(g) = \frac{1}{\sigma_g\sqrt{2\pi}}\exp\left(\frac{-(g-\mu_g)^2}{2\sigma_g^2}\right), \tag{2}$$

where μ_g is the average speed of all vehicles, and σ_g is the standard deviation.

Considering resource provisioning for DAG subtasks is conducted by vehicles that are located in the VC (i.e., within the coverage of the RSU), we utilize the notion of the *dwell time* to characterize vehicles' mobility. Specifically, considering that a contact event (i.e., V2V link formation) can happen between two vehicles as long as they have not left the VC, we define the dwell time of vehicle v_m in the VC as interval $[AT_m, DT_m]$, where AT_m and DT_m represent the arrival and departure time of v_m at and from the VC, respectively, between which vehicle v_m is available to offer its computation resource. Additionally, similar to [51], it is assumed that AT_m and DT_m are known. These values can be determined based on the predetermined routes of vehicles, such as taxis with fixed routes, and through statistical estimation methods that take into account factors, such as weather and traffic, to predict vehicles' routes.

D. Computation Offloading Model

We next model data transmissions over V2V links. The transmission rate $TR_{m,n}(t)$ from vehicle v_m to v_n at time t is given by

$$TR_{m,n}(t) = W\log_2\left(1 + \frac{P_m CS_{m,n}(t)}{\sigma^2}\right), \forall v_m, v_n \in \mathcal{V}, \tag{3}$$

where W is V2V channel bandwidth, P_m is the transmit power of vehicle v_m, $CS_{m,n}(t)$ is the V2V channel state between vehicles v_m and v_n at time t, and σ^2 is the noise power.[3]

[3]Similar to [38], we neglect the interference in V2V transmissions, which is realistic upon using interference suppression methods such as OFDMA, where different vehicles are assigned to different sub-channels.

Let $(x_m(t), y_m(t))$ denote the 2D coordinates of each vehicle v_m at time slot t, we then model the V2V channel state as an inverse power law given by [29]

$$CS_{m,n}(t) = \beta d_{m,n}(t)^{-2}, \forall v_m, v_n \in \mathcal{V}, \tag{4}$$

where β is the path loss at reference distance, and $d_{m,n}(t) = \sqrt{(x_m(t)-x_n(t))^2 + (y_m(t)-y_n(t))^2}$ (in meters) denotes the Euclidean distance between vehicles v_m and v_n at time slot t.

Subsequently, we introduce the notion of *ready time* which enables us to develop our scheduling methodology for DAG tasks by taking their sequential execution into account.

Definition 1 (Ready Time): Ready time RT_i indicates the time when all of the immediate predecessors of subtask b_i are completed/finished, which corresponds to the starting time of data transmission between b_j $(b_j \in \mathcal{P}_i)$ and the vehicle that processes b_i:

$$RT_i = \max_{b_j \in \mathcal{P}_i}\{AFT_j\}, \ b_i \in \mathcal{B}, \tag{5}$$

where AFT_j is the actual finish time of subtask b_j when it is practically executed on a vehicle. Considering the dynamic nature of VCs and opportunistic availability of vehicles, the ready time is leveraged to ensure the successful execution of all the predecessors of each subtask before planning on executing it [52].

Transmission Model: Let $TT_{i,m;j,n}$ denote the data transmission time associated with edge $e_{i,j}$ when subtasks b_i and b_j are allocated to vehicles v_m and v_n, respectively, which can be calculated as

$$TT_{i,m;j,n} = \begin{cases} \frac{c_{i,j}}{TR_{m,n}(RT_j)}, & m \neq n \\ 0, & m = n \end{cases}$$
$$\forall b_i, b_j \in \mathcal{B}, \ e_{i,j} \in \mathcal{E}, \ v_m, v_n \in \mathcal{V}, \tag{6}$$

where $c_{i,j}$ (in bits) is the transmission data size between subtasks b_i and b_j associated with directed edge $e_{i,j}$.

Computation Model: To model the scheduling of DAG subtasks, let $EST_{i,m}$ and $EFT_{i,m}$ denote the earliest start time, and finish time of processing of subtask b_i on vehicle v_m, respectively. We assume that virtual subtask b_0 is processed on the task owner (i.e., v_1) and its computation workload is zero, we thus have $EST_{0,1} = EFT_{0,1} = 0$.

Subsequently, for each subtask $b_i \in \mathcal{B}, i \neq 0$, the values of $EST_{i,m}$ and $EFT_{i,m}$ can be calculated recursively as follows:

$$EST_{i,m} = \max\left\{AVT_{i,m}, \overbrace{RT_i + \max_{b_j \in \mathcal{P}_i}\{TT_{j,n;i,m}\}}^{\text{(I)}}\right\},$$
$$\forall b_i \in \mathcal{B}, \ v_m \in \mathcal{V}, \tag{7}$$

where $AVT_{i,m}$ denotes the available time when vehicle v_m completes its latest assigned subtask and term (I) indicates the earliest arrival time of the required data for processing subtask b_i at vehicle v_m.

Furthermore, we consider heterogeneous computation capabilities across vehicles, where for each vehicle v_m, its computation capability is denoted by f_m (in CPU cycles per

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.

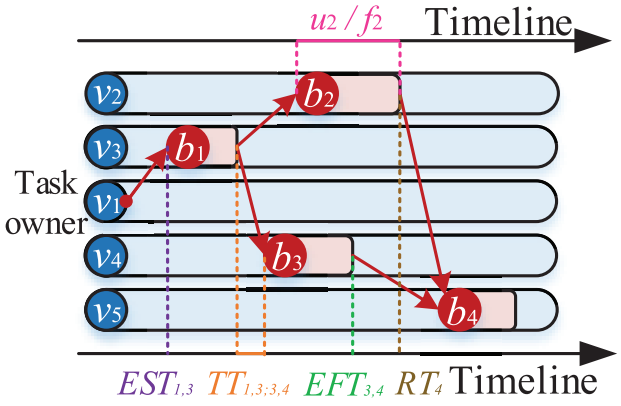


Fig. 3. A schematic of the timeline of our methodology for the DAG task allocation example depicted in Fig. 2, where virtual subtask b_0 is represented by a red node at the task owner, i.e., v_1.

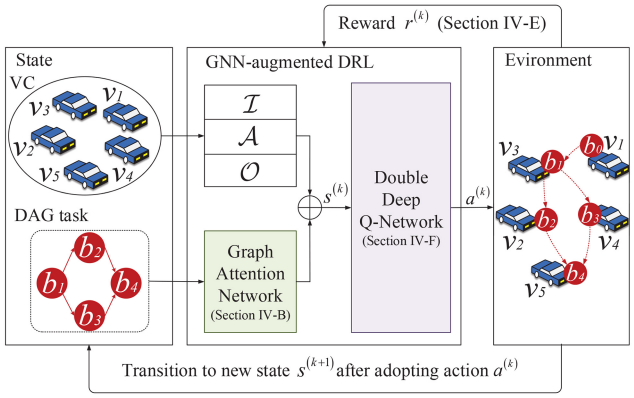


Fig. 4. A schematic of our GNN-augmented DRL methodology for VC-assisted DAG task scheduling.

second). As a result, the earliest finish time of processing subtask b_i on vehicle v_m is given by

$$EFT_{i,m} = EST_{i,m} + \frac{u_i}{f_m}, \ \forall b_i \in \mathcal{B}, v_m \in \mathcal{V}, \tag{8}$$

where u_i denotes the computation workload (in CPU cycles) of subtask b_i. A timeline diagram illustrating the procedure of our computation offloading method is depicted in Fig. 3.

E. Optimization Formulation

We capture the subtask-to-vehicle allocations through a set of binary indicators $\mathcal{I} = \{\xi_{i,m} \mid 0 \le i \le |\mathcal{B}|, \ 1 \le m \le |\mathcal{V}|\}$, where $\xi_{i,m} = 1$ denotes that subtask b_i is allocated to vehicle v_m, and $\xi_{i,m} = 0$ otherwise. Aiming to minimize the overall DAG task completion time, we formulate DAG task scheduling over the VC as the following Mixed Integer Programming (MIP):

$$\min_{\mathcal{I}} \ \max_{b_i \in \mathcal{B}, v_m \in \mathcal{V}} \{EFT_{i,m}\xi_{i,m}\}, \tag{9}$$

s. t. (7), (8),

$$\sum_{v_m \in \mathcal{V}} \xi_{i,m} = 1, \ b_i \in \mathcal{B}, \tag{C.1}$$

$$\xi_{i,m} \in \{0,1\}, \ b_i \in \mathcal{B}, \ v_m \in \mathcal{V}, \tag{C.2}$$

$$\bigcap_{b_i \in \mathcal{B}_m} \left[EST_{i,m}, EFT_{i,m}\right] = \varnothing, \ v_m \in \mathcal{V}, \tag{C.3}$$

$$EST_{i,m} \ge EFT_{j,n}, \ b_i \in \mathcal{B}, \ b_j \in \mathcal{P}_i, \ v_m, v_n \in \mathcal{V}, \tag{C.4}$$

$$\left[EST_{i,m}, EFT_{i,m}\right] \subset \left[AT_m, DT_m\right], \ b_i \in \mathcal{B}, \ v_m \in \mathcal{V}. \tag{C.5}$$

In (9), the objective function captures the sequential execution of DAG subtasks, where the maximum finish time of all subtasks indicates the overall DAG task completion time. Also, constraint (C1) guarantees that each subtask is allocated to only one vehicle, while (C2) restricts the value of the allocation indicator $\xi_{i,m}$ to be binary. Constraint (C3) ensures that a vehicle can only process one subtask at a time, where $\mathcal{B}_m = \{b_i \mid \xi_{i,m} = 1, \ 0 \le i \le |\mathcal{B}|\}$ denotes the set of subtasks processed on vehicle v_m. Constraint (C4) indicates that the processing of a subtask can not start until all of its predecessors are completed, (C5) guarantees the availability of computation resources of vehicles with respect to the vehicles' dwell times: the earliest start time and earliest finish time of executing each subtask b_i on vehicle v_m should between the arrival time and departure time of vehicle v_m in a VC.

It is known that MIP formulations similar to what we have in (9) are NP-hard [36]. Also, considering the sequential execution of DAG subtasks (i.e., different subtasks may be executed at different time slots), we need the prior knowledge of the V2V path loss and availability of vehicles' computation resources, obtaining of which is cumbersome in practice. As a result, to tackle these challenges, we propose a GNN-augmented DRL scheme, named GA-DRL, to efficiently find near-optimal solutions for (9).

IV. GNN-Augmented DRL (GA-DRL) for DAG Task Scheduling Over Dynamic VCs

In this section, we first provide an overview of our GA-DRL methodology and the challenges we aim to address. We then tailor a GAT module for extracting features of subtasks. Subsequently, the VC-assisted DAG task scheduling is modeled as an MDP consisting of the state space, action space, and reward. Finally, we utilize a DDQN architecture to tackle (9) and discuss its training procedure.

A. GA-DRL Overview and Challenges

1) GA-DRL Overview: Our method takes a different approach from traditional DRL methods developed for task scheduling [16], [18], [25], which only consider predetermined states, such as computation workload, data size, and number of subtask predecessors/successors. Instead, we propose a GNN-augmented DRL approach that automatically learns distinctive subtask features and creates assignments between subtasks and vehicles. In particular, as shown in Fig. 4, the features of subtasks are acquired through a GAT module, rather than being predetermined.

Our GA-DRL conducts subtask-to-vehicle allocations through a sequence of decision steps. At each decision step

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SUMMARY OF THE NOTATIONS USED IN GA-DRL DESIGN

$h_i^{(0)}$	Raw feature of subtask b_i
$\mathcal{N}_i$	Neighbor set of subtask b_i
$\mathcal{N}_i^{-1}$	Inverse neighbor set of subtask b_i
$h_i^{(\ell)}$	Result feature of subtask b_i at iteration ℓ
$\alpha_{i,j}^{(\ell)}$	Attention coefficient between subtasks b_i and b_j at iteration ℓ
$W^{(\ell)}$	Trainable weight matrix of GAT at iteration ℓ
$A^{(\ell)}$	Trainable feedforward neural network at iteration ℓ
$\mathcal{L}^{\text{rank}}$	Subtask scheduling priority list

k, the DRL agent functioning at the RSU diligently collects relevant data on the system *state* $s^{(k)}$, which includes the extracted features of current subtask obtained by GAT, as well as the parameters of the vehicles describing their dynamics and heterogeneity. DRL agent then feeds state $s^{(k)}$ to a DDQN. The objective of DDQN is to effectively assign subtasks to vehicles by determining the best course of *action* $a^{(k)}$. To this end, DDQN evaluates the value of each state-action combinations, and conducts a subtask-to-vehicle allocation $a^{(k)}$, which moves the system to the next state $s^{(k+1)}$. Finally, the DRL agent receives a *reward* $r^{(k)}$, that aids in the training of a deep learning model. This, in turn, enhances the agent's ability to take better actions over time.

2) Main Challenges: When applying GA-DRL to the VC-assisted DAG task scheduling, there are two main challenges that need to be tackled.

(1) *Feasibility of allocation decisions:* Unlike static computing environments that have stable, fully-connected computing servers [3], [4], [6], the dynamics of VC's resources can significantly impact the execution of DAG subtasks. This is captured by constraint (C5) in (9), satisfying of which guarantees the time-interval of processing subtask b_i on vehicle v_m to be within the availability time window of v_m in the VC. Ensuring that subtask-to-vehicle allocation decisions are feasible (specifically, meeting constraint (C5)) can be difficult since neural networks, which are the building blocks of DRL methods and guide their actions, often lack a module to filter out infeasible actions.

(2) *Generalizability of designed GNN:* Efficient inductive learning is a key feature of GAT [31], which makes it suitable for using with previously unseen graph topologies. However, it can be difficult to ensure that the GNN model is applicable to various DAG tasks, as each task has its own unique topology and interdependence among subtasks. To overcome this challenge, we must carefully encode the information of each DAG task's topology to achieve meaningful results when combined with our later developed GAT.

Table II summarizes the major notations used in this section.

B. Graph Neural Network

In this subsection, we explain the structure of GNNs and how we use a multi-head GAT to extract distinctive features of subtasks. Our GAT incorporates a two-way aggregation method that considers the topological information of both predecessors and successors of each subtask. To further enhance the adaptability of our GAT to new DAG tasks, we utilize a ranking-based sampling technique.

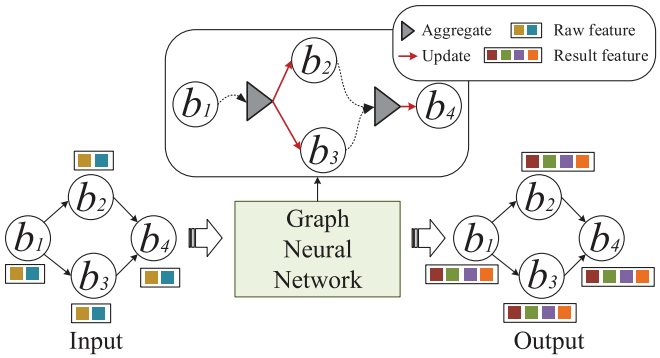


Fig. 5. A schematic of architecture of GNN for extracting the features of the target tracking application depicted in Fig. 1. The colored squares in the diagram correspond to different features.

1) Architecture of GNNs: A schematic of the architecture of GNN is depicted in Fig. 5, where the GNN takes raw features of subtasks as its input and generates result features containing topological information of the DAG task. Specifically, GNN utilizes an **Aggregate** function to accumulate the topological information of the neighbors of each subtask. The accumulated information is then modified through a nonlinear **Update** function. This procedure is repeated L times to create the result feature for each subtask.

Raw feature of each subtask: Similar to conventional DRL methods [20], [21], [22], [23], [24], which rely on human-selected information to define DAG subtasks, we also define the raw feature[4] of each subtask b_i as

$$h_i^{(0)} = \{u_i, \overline{c_i}, |\mathcal{P}_i|, |\mathcal{S}_i|\}, \ b_i \in \mathcal{B}, \ b_j \in \mathcal{S}_i, \tag{10}$$

where u_i is the computation workload of subtask b_i, and $\overline{c_i} = \sum_{b_j \in \mathcal{S}_i} \frac{c_{i,j}}{|\mathcal{S}_i|}$ indicates the average transmission data size associated with edges $e_{i,j}$, $b_j \in \mathcal{S}_i$. Also, $|\mathcal{P}_i|$ and $|\mathcal{S}_i|$ represent the number of predecessors and successors of subtask b_i, respectively.

Neighbor set of each subtask: Considering that DAG subtasks are executed sequentially, we define $\mathcal{N}_i$ as the neighbor set of each subtask b_i which includes all of its predecessors as well as b_i itself; mathematically

$$\mathcal{N}_i = \{b_j \mid e_{j,i} \in \mathcal{E}\} \cup \{b_i\}. \tag{11}$$

Through an iterative process involving the use of **Update** and **Aggregate** functions, GNN obtains the result feature of each subtask b_i. Mathematically, at each iteration ℓ, we have

$$h_i^{(\ell+1)} = \textbf{Update}\Big(\textbf{Aggregate}(\{h_j^{(\ell)} | b_j \in \mathcal{N}_i\})\Big). \tag{12}$$

Then, through L iterations, the GNN derives the final result feature for each subtask, denoted by $h_i^{(L)}$. This feature incorporates both the raw feature of each subtask (i.e., at $\ell =$

[4]Super-index 0 is used to capture that these are initial features of the subtask, which are later processed and enhanced through GNN.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.

$0; h_i^{(0)}$), as well as the topological information of neighboring subtasks (i.e., $\mathcal{N}_i$) within the DAG task.

Hereafter, we detail the **Aggregate** and the **Update** functions designed to extract features of DAG subtasks.

2) Multi-Head GAT: Considering that the subtasks involved in $\mathcal{N}_i$ have different computation workloads, transmission data sizes and interdependencies, we employ an attention mechanism, which is inspired by [31] to assign diverse weights to subtasks with the aim of enhancing the quality of the extracted information from subtasks. Specifically, at each iteration ℓ, we define an attention-based aggregation function called $\textbf{Aggregate}^{\mathsf{at}}$ as

$$\textbf{Aggregate}^{\mathsf{at}}\Big(\{h_j^{(\ell)} | b_j \in \mathcal{N}_i\}\Big) = \sum_{b_j \in \mathcal{N}_i} \alpha_{i,j}^{(\ell)} W^{(\ell)} h_j^{(\ell)}, \quad (13)$$

where $W^{(\ell)}$ is a trainable weight matrix at iteration ℓ, and $\alpha_{i,j}^{(\ell)}$ is a normalized attention coefficient at iteration ℓ, which measures the relative importance of subtask b_j to subtask b_i as follows:

$$\alpha_{i,j}^{(\ell)} = \frac{\exp\Big(A^{(\ell)}\Big[W^{(\ell)} h_i^{(\ell)} || W^{(\ell)} h_j^{(\ell)}\Big]\Big)}{\sum_{b_{j'} \in \mathcal{N}_i} \exp\Big(A^{(\ell)}\Big[W^{(\ell)} h_i^{(\ell)} || W^{(\ell)} h_{j'}^{(\ell)}\Big]\Big)}. \quad (14)$$

In (14), $A^{(\ell)}$ is a trainable vectors at iteration ℓ, and $\cdot||\cdot$ denotes the vector concatenation.

Further, in order to enhance the effectiveness of GAT's learning process, we propose to use a multi-head GAT, where different attention heads[5] learn to give weights/importances for different attributes of each subtask. Let Z denote the total number of heads. Each attention head, denoted by z will individually aggregate topological information of subtasks, in conjunction with other attention modules. The multi-head attention-based aggregation function called $\textbf{Aggregate}^{\mathsf{mat}}$ can be then formulated as

$$\textbf{Aggregate}^{\mathsf{mat}}\Big(\{h_j^{(\ell)} | b_j \in \mathcal{N}_i\}\Big) = \frac{1}{Z} \sum_{z=1}^{Z} \left(\sum_{b_j \in \mathcal{N}_i} \alpha_{i,j}^{(\ell)(z)} W^{(\ell)(z)} h_j^{(\ell)} \right), \quad (15)$$

where $W^{(l)(z)}$ denote the trainable weight matrix of attention head z at iteration l, and

$$\alpha_{i,j}^{(\ell)(z)} = \frac{\exp\Big(A^{(\ell)(z)}\Big[W^{(\ell)(z)} h_i^{(\ell)} || W^{(\ell)(z)} h_j^{(\ell)}\Big]\Big)}{\sum_{b_{j'} \in \mathcal{N}_i} \exp\Big(A^{(\ell)(z)}\Big[W^{(\ell)(z)} h_i^{(l)} || W^{(\ell)(z)} h_{j'}^{(\ell)}\Big]\Big)}, \quad (16)$$

indicates a normalized attention coefficient, which measures the relative importance of subtask b_j to subtask b_i of attention head z at iteration l.

It is worth mentioning that once we acquire the trainable parameters $A^{(\ell)(z)}$ and $W^{(\ell)(z)}$, the feature of subtask b_i (i.e., $h_i^{(L)}$), can be obtained by aggregating the information of neighboring nodes in $\mathcal{N}_i$ following L iterations. Due to the localized nature of this process (i.e., moving from one subtask to another and pooling the information from the neighboring subtasks), the GAT model is effective even if the DAG topology has not been seen during its training process, eliminating the necessity to relearn the corresponding parameters for newly arriving DAG task topologies.

To better suit our problem, we aim to modify the $\textbf{Aggregate}^{\mathsf{mat}}$ function defined in (15) through developing a two-way aggregation for the multi-head GAT. This approach takes into consideration the predecessors and successors of each subtask, which helps to aggregate topological information in a more effective manner.

3) Two-Way Aggregation: To execute DAG subtasks, capturing the conditions of predecessors and successors of each subtasks are equally important. As a result, we develop a two-way aggregation approach that utilizes two different types of attention heads. This approach involves using the inverse neighbor set $\mathcal{N}_i^{-1}$ of each subtask b_i, which includes all of its successors and b_i itself

$$\mathcal{N}_i^{-1} = \{b_j \mid e_{i,j} \in \mathcal{E}\} \cup \{b_i\}. \quad (17)$$

At each iteration ℓ, half of the attention heads from Z are then allocated to collect topological information from the neighboring subtasks, while the remaining half is utilized to gather topological information from the inverse neighbor set, which leads to the modification of (15) to

$$\textbf{Aggregate}^{\mathsf{mat}}\Big(h_j^{(\ell)}\Big) = \frac{1}{Z}\left[\sum_{z=1}^{Z/2}\left(\sum_{b_j \in \mathcal{N}_i} \alpha_{i,j}^{(\ell)(z)} W^{(\ell)(z)} h_j^{(\ell)}\right) + \sum_{z=Z/2}^{Z}\left(\sum_{b_j \in \mathcal{N}_i^{-1}} \alpha_{i,j}^{(\ell)(z)} W^{(\ell)(z)} h_j^{(\ell)}\right)\right]. \quad (18)$$

We then aim to further modify the $\textbf{Aggregate}^{\mathsf{mat}}$ function defined in (18). This makes our GAT module different from other existing GNNs [34], [35], [36], [37]: we do not consider all the neighbors of a given subtask to accumulate topological information. Instead, we opt for a weighted sample of neighbors, based on their scheduling priority. This approach allows our GAT to be more generalizable to unseen DAG task topologies. We next describe this approach.

4) Ranking-Based Sampling: We first devise an approach to prioritize scheduling of subtasks based on their ranking value. By employing a recursive method, we determine the ranking value of each subtask b_i labeled as $rank_i$ as follows:

$$rank_i = \max_{b_j \in \mathcal{P}_i} \{rank_j + \overline{u_j} + \overline{c_{j,i}}\}, \ b_i \in \mathcal{B}, \quad (19)$$

where $\overline{u_j}$ is the average execution cost of subtask $b_j, b_j \in \mathcal{P}_i$, which is given by

$$\overline{u_j} = \frac{\sum_{m=1}^{|\mathcal{V}|} u_j / f_m}{|\mathcal{V}|}, \ v_m \in \mathcal{V}, \quad (20)$$

[5]Considering that each subtask has different attributions, e.g., computation workload, transmission data size and interdependency, we employ a multi-head GAT to enhance the quality of the extracted information from subtasks by obtaining weights/importances for different attributes of each subtask.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.

and $\overline{c_{j,i}}$ denotes the average transmission cost associated with edge $e_{j,i}, b_j \in \mathcal{P}_i$ at the beginning (i.e., at time slot 0), which is given by

$$\overline{c_{j,i}} = \frac{\sum_{m=1}^{|\mathcal{V}|}\sum_{n=1}^{|\mathcal{V}|} c_{j,i}\Psi\big(PL\big(d_{m,n}(0)\big)\big)}{|\mathcal{V}|^2}. \tag{21}$$

Assuming $rank_0 = 0$ for virtual subtask b_0, we maintain a subtask scheduling priority list $\mathcal{L}^{\mathsf{rank}}$ as

$$\mathcal{L}^{\mathsf{rank}} = \{b_i \succ b_j \mid b_i, b_j \in \mathcal{B},\ rank_i < rank_j\}, \tag{22}$$

where the preference relation $b_i \succ b_j$ indicates that subtask b_i has a higher scheduling priority compared with subtask b_j due to a lower value of $rank_i$.[6]

Finally, we define $\mathcal{N}_i^{\mathsf{rank}}$ as a ranking-based neighbor set of subtask b_i which contains the subtasks sampled from $\mathcal{N}_i$. The sampling probability/weight of subtask b_j from $\mathcal{N}_i$ to be included in $\mathcal{N}_i^{\mathsf{rank}}$, denoted by p_j, is calculated as

$$p_j = \frac{\exp\big(rank_j\big)}{\sum_{b_{j'}\in\mathcal{P}_i}\exp\big(rank_{j'}\big)}. \tag{23}$$

This weighted subtask sampling method leads to improving the generalizability of our method. Specifically, similar to dropout mechanism [19], within the training process of GA-DRL, we can consider sampling a set of subtasks (i.e., nodes) as reducing the resolution of learning, which leads to reducing the bias and overfitting of the GA-DRL model to the training DAG structure. Consequently, our multi-head GAT will aggregate only the topological messages conveyed by the sampled subtasks. By intentionally losing topological information from unsampled subtasks, our GAT model becomes less sensitive to topological changes in DAG tasks upon being deployed during the execution time. Note that subtask sampling is done with replacement if the sample size is larger than the size of $\mathcal{N}_i$.

Aggregate function: By integrating aforementioned methodologies, our designed $\mathbf{Aggregate}^{\mathsf{mat}}$ function not only enables the enhancement of the quality of the extracted information from subtasks by considering a two-way multi-head attention-based aggregation, but also improves generalizability by considering a ranking-based sampling; mathematically

$$\mathbf{Aggregate}^{\mathsf{mat}}\Big(h_j^{(\ell)}\Big) = \frac{1}{Z}\Bigg[\sum_{z=1}^{Z/2}\Big(\sum_{b_j\in\mathcal{N}_i^{\mathsf{rank}}}\alpha_{i,j}^{(\ell)(z)}\,W^{(\ell)(z)}h_j^{(\ell)}\Big) + \sum_{z=Z/2}^{Z}\Big(\sum_{b_j\in\mathcal{N}_i^{-\mathsf{rank}}}\alpha_{i,j}^{(\ell)(z)}\,W^{(\ell)(z)}h_j^{(\ell)}\Big)\Bigg], \tag{24}$$

where $\mathcal{N}_i^{-\mathsf{rank}}$ is the inverse ranking-based neighbor set of subtask b_i sampled from the $\mathcal{N}_i^{-1}$ using a similar sampling method described in (23).

Update function: After receiving aggregated topological information in (24), we apply the Exponential Linear Unit activation (ELU) [48] in the **Update** function. Finally, combining the aforementioned **Aggregate** and **Update** functions, we can express (12) as

$$h_i^{(\ell+1)} = \mathbf{ELU}\Bigg(\frac{1}{Z}\Bigg[\sum_{z=1}^{Z/2}\Bigg(\sum_{b_j\in\mathcal{N}_i^{\mathsf{rank}}}\alpha_{i,j}^{(\ell)(z)}\,W^{(\ell)(z)}h_j^{(\ell)}\Bigg) + \sum_{z=Z/2}^{Z}\Bigg(\sum_{b_j\in\mathcal{N}_i^{-\mathsf{rank}}}\alpha_{i,j}^{(\ell)(z)}\,W^{(\ell)(z)}h_j^{(\ell)}\Bigg)\Bigg]\Bigg). \tag{25}$$

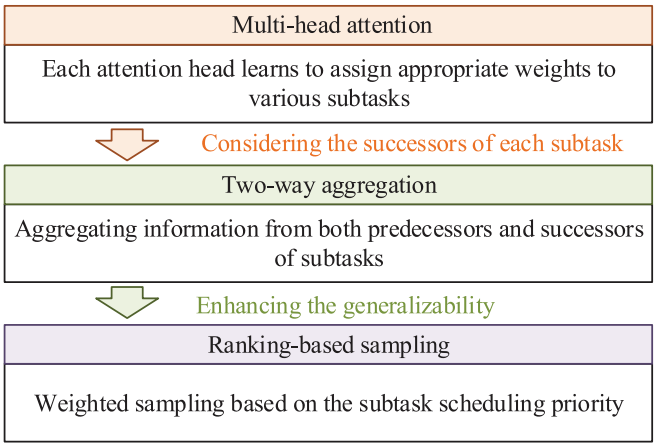


Fig. 6. A flow chart of the relationships between different components of our methodology in Section IV-B.

A flow chart of the relationships between the components developed for **Aggregate** function is shown in Fig. 6. Algorithm 1 outlines the procedure of our GAT module with learned parameters $W^{(\ell)(z)}$ and $A^{(\ell)(z)}$, which will be optimized later in conjunction with DDQN parameters. The computational complexity of a single GAT attention head z used in Algorithm 1 is given by $\mathcal{O}(L|\mathcal{B}||h^{(0)}||h^{(L)}| + 2L|\mathcal{E}||h^{(L)}|)$, where $|h^{(0)}|$ and $|h^{(L)}|$ are the dimension of raw and result features, respectively. $|\mathcal{B}|$ and $|\mathcal{E}|$ represent the numbers of subtasks and directed edges in the DAG task, respectively, highlighting that the attention mechanism's process of calculating attention coefficients between each pair of subtasks (i.e., nodes) will linearly grow with the number of nodes and edges in the DAG task structure (i.e., high DAG complexity). As compared to using predetermined handcrafted features (e.g., conventional DRL-based techniques, such as DRLOSM, which we study in Section V), due to the extraction of more informative topological structure information from DAG tasks, our proposed multi-head GAT can make more effective subtask-to-vehicle allocation decisions with marginally higher complexity (as verified through simulations, e.g., Table III in Section V).

We next formulate DAG task scheduling as an MDP with state, action, and reward representation. Specifically, the VC-assisted DAG task scheduling based on an MDP can be modeled as a sequence of tuples $(s^{(k)}, a^{(k)}, r^{(k)}, s^{(k+1)})$, where k denotes the decision step. At each decision step k, MDP is in *state* $s^{(k)}$ containing the extracted features of the current subtask obtained by GAT, as well as the parameters

[6]Our current ranking method for DAG subtasks relies on heuristics, which may limit the GNN-augmented DRL algorithm's ability. We plan to address this issue by exploring alternative methods for determining the scheduling priority of DAG subtasks using DRL in the future.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.

TABLE III
PERFORMANCE COMPARISON UNDER DIFFERENT METRICS FOR MOLECULAR DYNAMICS CODE DAG

Metric	HEFT	MGA	DRLOSM	GA-DRL
Task completion time	24.12	22.31	19.84	16.47
Algorithm running time	2.54	8.33	2.13	2.84

Algorithm 1: Ranking-Based Two-Way Multi-Head GAT

Input: Graph $\mathcal{G} = (\mathcal{B}, \mathcal{E})$; raw features of subtasks $\{h_i^{(0)}, b_i \in \mathcal{B}\}$; iteration number L; head number Z; weight $\{W^{(\ell)(z)}, \forall \ell \in \{1, \ldots, L\}, \forall z \in \{1, \ldots, Z\}\}$; network $\{A^{(\ell)(z)}, \forall \ell \in \{1, \ldots, L\}, \forall z \in \{1, \ldots, Z\}\}$
Output: Result features $h_i^{(L)}$ for all subtasks $b_i \in \mathcal{B}$
Obtain $\mathcal{L}^{\mathsf{rank}}$ by (19)-(23)
for $i = 1 \cdots |\mathcal{B}|$ **do**
  //Ranking-based sampling
  Weighted sample a fixed-size set $\mathcal{N}_i^{\mathsf{rank}}$ of neighbors in $\mathcal{N}_i$ according to (11) and (23)
  for $\ell = 1 \cdots L$ **do**
    //Multi-head attention
    for $z = 1 \cdots Z/2$ **do**
      for $j = 1 \cdots \mathcal{N}_i^{\mathsf{rank}}$ **do**
        Calculate $\alpha_{i,j}^{(\ell)(z)}$ by (14)
      end for
    end for
    //Two-way aggregation
    Weighted sample a fixed-size set $\mathcal{N}_i^{-\mathsf{rank}}$ of neighbors in $\mathcal{N}_i^{-1}$ according to (17) and (23)
    for $z = Z/2 \cdots Z$ **do**
      for $j = 1 \cdots \mathcal{N}_i^{-\mathsf{rank}}$ **do**
        Calculate $\alpha_{i,j}^{(\ell)(z)}$ by (14)
      end for
    end for
    Use (24) to aggregate the massages passed by neighbors in $\mathcal{N}_i^{\mathsf{rank}}$ as well as $\mathcal{N}_i^{-\mathsf{rank}}$
    Updating the accumulative features using (25)
  end for
end for

related to the configuration of vehicles (as will be discussed later, this state will be fed to the DRL agent). Subsequently, the MPD transitions from state $s^{(k)}$ through taking an *action* $a^{(k)}$ indicating the vehicle index to which the current subtask is allocated (as will be discussed later, this action will be taken by the DRL agent using our developed DDQN). As a result, the MDP will produce/observe a *reward* $r^{(k)}$ and transition to the next state *state* $s^{(k+1)}$.

C. State Representation

The result feature of each subtask b_i, denoted by $h_i^{(L)}$, is generated through consecutive L iterations in (25). We assume subtasks to vehicles assignments through a series of decision steps indexed by k, at each decision step k, let subtask $b_{\tau(k)}$ be the *current subtask* waiting to be allocated to a vehicle, where $\tau(k)$ indicates the subtask's index at position k in $\mathcal{L}^{\mathsf{rank}}$. We define the system state $s^{(k)}$ as follows:

$$s^{(k)} = \left\{h_{\tau(k)}^{(L)}, \mathcal{I}^{(k-1)}, \mathcal{A}^{(k)}, \mathcal{O}^{(k)}\right\}, \tag{26}$$

where $\mathcal{A}^{(k)} = \{avail_1, avail_2, \ldots, avail_{|\mathcal{V}|}\}$ signifies the set of availability indicators at the moment of decision step k. Here, $avail_m = 1$ indicates that vehicle v_m is available to offer its computation resource or to process current subtask, and $avail_m = 0$ otherwise, thus capturing the temporal availability of resources within the VC. Furthermore, $\mathcal{O}^{(k)} = \{(x_m, y_m) \mid v_m \in \mathcal{V}\}$ represents the instantaneous locations of vehicles at decision step k, enabling us to adapt our solution to the mobility of vehicles.

D. Action Space

During each decision step k, we need to determine which vehicle should be assigned to each subtask based on the system state $s^{(k)}$ and subtask scheduling priority list $\mathcal{L}^{\mathsf{rank}}$. In particular, at decision step k, for current subtask $b_{\tau(k)}$, action $a^{(k)}$ is defined as

$$a^{(k)} \in \{1, 2, \ldots, |\mathcal{V}|\}, \tag{27}$$

where $a^{(k)} = 1$ implies that current subtask $b_{\tau(k)}$ is processed locally on task owner v_1, and $a^{(k)} \in \{2, \ldots, |\mathcal{V}|\}$ implies that current subtask $b_{\tau(k)}$ is allocated to other vehicles for a faster execution.

E. Reward Design

At decision step k, given state $s^{(k)}$, we associate performing action $a^{(k)}$ for allocating of current subtask $b_{\tau(k)}$ to an immediate reward $r^{(k)}$ leveraged to evaluate the quality of action $a^{(k)}$. We define the reward $r^{(k)}$ as the decrease in the *EFT* of all subtasks as

$$r^{(k)} = \underbrace{\max_{b_i \in \mathcal{B}, v_m \in \mathcal{V}} \{EFT_{i,m}^{(k-1)}\}}_{\text{(I)}} - \underbrace{\max_{b_i \in \mathcal{B}, v_m \in \mathcal{V}} \{EFT_{i,m}^{(k)}\}}_{\text{(II)}}, \tag{28}$$

where term (I) and (II) denote the maximum DAG task completion time before and after scheduling the current subtask, respectively. We next demonstrate the rationality of reward function introduced above.

Rational of the Choice of Reward: Let K denote the total number of decision steps. According to (28), the discounted cumulative reward can be calculated as

$$\begin{aligned} R &= \sum_{k=1}^{K} \gamma_1^k r^{(k)} \\ &= \sum_{k=1}^{K} \gamma_1^k \left(\max_{b_i \in \mathcal{B}, v_m \in \mathcal{V}} \left\{ EFT_{i,m}^{(k-1)} \right\} - \max_{b_i \in \mathcal{B}, v_m \in \mathcal{V}} \left\{ EFT_{i,m}^{(k)} \right\} \right), \end{aligned} \tag{29}$$

where γ_1 is the discount factor. Assuming $\gamma_1 = 1$ for simplicity, since at decision step k, we determine the allocation of only the current subtask $b_{\tau(k)}$ according to scheduling

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.

priority list $\mathcal{L}^{\mathsf{rank}}$, we have $K = |\mathcal{L}^R| = |\mathcal{B}|$. Thus, (29) can be rewritten as

$$\begin{aligned} R &= \sum_{k=1}^{K} \left(\max_{v_m \in \mathcal{V}} \left\{ EFT_{\tau(k-1),m} \right\} - \max_{v_m \in \mathcal{V}} \left\{ EFT_{\tau(k),m} \right\} \right) \\ &= \left(\max_{v_m \in \mathcal{V}} \left\{ EFT_{\tau(0),m} \right\} - \max_{v_m \in \mathcal{V}} \left\{ EFT_{\tau(1),m} \right\} \right. \\ &\quad \left. + \max_{v_m \in \mathcal{V}} \left\{ EFT_{\tau(1),m} \right\} + \cdots - \max_{v_m \in \mathcal{V}} \left\{ EFT_{\tau(K),m} \right\} \right) \\ &= -\left(\max_{v_m \in \mathcal{V}} \left\{ EFT_{\tau(K),m} \right\} - \max_{v_m \in \mathcal{V}} \left\{ EFT_{\tau(0),m} \right\} \right), \end{aligned} \tag{30}$$

where we define $b_{\tau(0)}$ as the virtual subtask with $\max_{v_m \in \mathcal{V}} \{EFT_{\tau(0),m}\} = 0$. The last result in (30) (i.e., term $-\max_{v_m \in \mathcal{V}} \{EFT_{\tau(K),m}\}$) indicates that maximizing the cumulative reward is consistent with minimizing the task completion time given in (9).

Hereafter, in order to solve the above mentioned MDP, we resort to a DDQN, which adopts the action (i.e., subtask-to-vehicle allocation) at each decision step yielding the largest Q-value (i.e., state-action value) prior to DAG task scheduling over dynamic VCs.

F. Double Deep Q-Network

1) Deep Q-Network: We first describe DQN methodology [49], which paves the way for DDQN. In DQN, we have two Deep Neural Networks (DNNs) called *predict* Q-network $Q(s, a; \theta^{\mathsf{p}})$ and *target* Q-network $Q(s, a; \theta^{\mathsf{t}})$. Particularly, θ^{p} and θ^{t} are the vectors of weights/parameters of DNNs, and s and a denote the state and action, respectively.

Predict Q-value: At each decision step k, given state $s^{(k)}$, using the predict Q-network, the DRL agent first estimates/predicts the Q-value $Q(s^{(k)}, a; \theta^{\mathsf{p}})$ of all actions $a = 1, 2, \ldots, |\mathcal{V}|$, where $s^{(k)}$ consists of the extracted feature of current subtask $b_{\tau(k)}$ and vehicles' parameters given in (26). Q-value is a measure of the quality of the action: a higher Q-value is an indicator to a better action.

Action selection: The DRL agent then performs an action $a^{(k)}$ using a max mathematical estimator as follow

$$a^{(k)} = \mathrm{argmax}_a \, Q\left(s^{(k)}, a; \theta^{\mathsf{p}}\right), a \in \{1, 2 \cdots |\mathcal{V}|\}. \tag{31}$$

The DRL agent then receives a reward $r^{(k)}$ computed by (28).

Target Q-value: The system subsequently transits to the next state $s^{(k+1)}$, and the DRL agent resorts to target Q-network for calculating the target Q-value of state $s^{(k)}$, denoted by $\mathsf{y}^{(k)}$:

$$\mathsf{y}^{(k)} = r^{(k)} + \gamma_2 \underbrace{Q\left(s^{(k+1)}, \underbrace{\mathrm{argmax}_a \, Q\left(s^{(k+1)}, a; \theta^{\mathsf{t}}\right)}_{\text{(I)}}; \theta^{\mathsf{t}} \right)}_{\text{(II)}}, \quad a \in \{1, 2 \cdots |\mathcal{V}|\}. \tag{32}$$

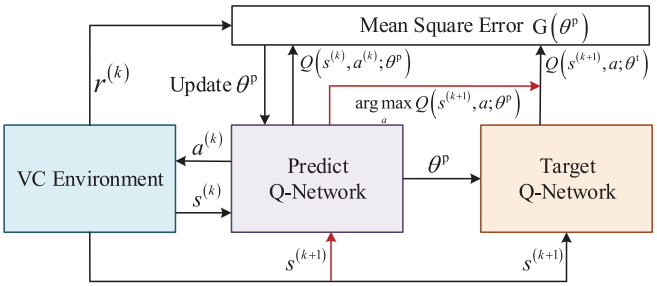


Fig. 7. System-level description of our double DQN methodology.

To obtain the parameter θ^{p}, the mean square error, denoted by $\mathsf{G}(\theta^{\mathsf{p}})$, is used with discount factor γ_2 as follows

$$\mathsf{G}(\theta^{\mathsf{p}}) = \frac{1}{2}\left[\mathsf{y}^{(k)} - Q\left(s^{(k)}, a^{(k)}; \theta^{\mathsf{p}}\right)\right]^2, a \in \{1, 2 \cdots |\mathcal{V}|\}. \tag{33}$$

Also, the weights of the target network θ^{t} are periodically copied from the predict network θ^{p}.

2) Double Deep Q-Network: In standard DQN, the max operator employs the same values to both select (i.e., term (I) in (32)) and evaluate (i.e., term (II) in (32)) an action. This implies that the Q-values are updated based on estimated future rewards, rather than actual rewards. Thus, there is a risk of overestimating Q-values, especially when the estimates are based on an inaccurate model of the environment. To prevent this, we resort to DDQN [50] aiming at separating the action selection from the action evaluation. In DDQN, the action with the maximum Q-value is selected using the predict network, and the Q-value for this action is evaluated using the target network. In particular, DDQN uses the same approach for predicting Q-value and selecting action as DQN. However, it uses a different target network update rule detailed next.

Target Q-value in DDQN: The target value of state $s^{(k)}$ in DDQN (see the red line shown in Fig. 7), denoted by $\mathsf{y}^{(k)}_{\mathsf{Double}}$ is changed from (32) to

$$\mathsf{y}^{(k)}_{\mathsf{Double}} = r^{(k)} + \gamma_2 Q\left(s^{(k+1)}, \arg\max_a Q\left(s^{(k+1)}, a, \theta^{\mathsf{p}}\right); \theta^{\mathsf{t}}\right), \quad a \in \{1, 2 \cdots |\mathcal{V}|\}, \tag{34}$$

where the action a is conducted by predict Q-network $Q(s, a; \theta^{\mathsf{p}})$ and the target value of state $s^{(k)}$ is evaluated by target Q-network $Q(s, a; \theta^{\mathsf{t}})$, which avoids the risk of overestimating the quality of the taken actions [50]. Finally, the mean square error for training predict Q-network $Q(s, a; \theta^{\mathsf{p}})$ is modified from (33) to

$$\mathsf{G}(\theta^{\mathsf{p}}) = \frac{1}{2}\left[\mathsf{y}^{(k)}_{\mathsf{Double}} - Q\left(s^{(k)}, a^{(k)}; \theta^{\mathsf{p}}\right)\right]^2, a \in \{1, 2 \cdots |\mathcal{V}|\}. \tag{35}$$

Using which the parameter θ^{p} is obtained, the weights of the target network θ^{t} are then periodically copied from the predict network θ^{p}.

G. Training Process

We consider training of DRL through a series of *episodes*, where each episode contains total of K sequential decision

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.

steps. At each decision step k, DRL agent generates a pair of observation $(s^{(k)}, a^{(k)}, r^{(k)}, s^{(k+1)})$. An episode is considered to be complete when a vehicle is assigned the subtask with the lowest scheduling priority, which is listed in the last position of $\mathcal{L}^{\mathsf{rank}}$ (i.e, $K = |\mathcal{B}|$).

1) Q-Network Training: Based on policy gradient algorithm [40], predict Q-network $Q(s, a; \theta^{\mathsf{p}})$ is trained by iteratively tuning the weights θ^{p} at each decision step k through minimizing the mean square error given in (35) as follows:

$$\theta^{\mathsf{p}} \leftarrow \theta^{\mathsf{p}} - \mu \frac{\partial \mathsf{G}(\theta^{\mathsf{p}})}{\partial \theta^{\mathsf{p}}} \tag{36}$$

where μ is the tunable learning rate. As for the target Q-network $Q(s, a; \theta^{\mathsf{t}})$, θ^{t} is copied from θ^{p} at beginning, and θ^{t} will be iteratively updated to θ^{p} after conducting some iterations (5 decision steps in our simulations).

We adopt a ϵ-greedy policy to select action, in which the DRL agent probabilistically explores the actions which have not been adopted yet instead of an action with the maximum Q-value in (31). Also, we leverage a replay buffer $\mathcal{R}$ to store the sequence of $(s^{(k)}, a^{(k)}, r^{(k)}, s^{(k+1)})$ obtained through decision steps k. In particular, the gradient in (36) is obtained by selecting mini-batches of data from the reply buffer. At each decision step k, we consider the feasibility of actions for the current subtask $b_{\tau(k)}$. Throughout the training process of GA-DRL, actions that fail to meet constraint (C5) are considered infeasible. Subsequently, we utilize action mask technique [47] to prevent the DDQN from performing infeasible actions. This is achieved by assigning a significantly negative Q-value to any infeasible action, thereby guaranteeing that only feasible actions are selected.

2) GAT Training: The state $s^{(k)}$ which consists of the extracted features of current subtask $b_{\tau(k)}$ is obtained from the GAT with parameters $\mathcal{W} = \{W^{(\ell)(z)} \mid 1 \le \ell \le L,\ 1 \le z \le Z\}$ and $\mathcal{A} = \{A^{(\ell)(z)} \mid 1 \le \ell \le L,\ 1 \le z \le Z\}$. Thus, we can rewrite the right hand-side of (35) as

$$\left[r^{(k)} + \gamma_2 Q\left(s^{(k+1)}(\mathcal{W}, \mathcal{A}), \arg\max_{a^*} Q\left(s^{(k+1)}, a^*, \theta^{\mathsf{p}}\right); \theta^{\mathsf{t}}\right) - Q\left(s^{(k)}(\mathcal{W}, \mathcal{A}), a^{(k)}; \theta^{\mathsf{p}}\right)\right]^2, a \in \{1, 2 \cdots |\mathcal{V}|\}, \tag{37}$$

which indicates that parameters $\mathcal{W}$, $\mathcal{A}$ and θ^{p} are trained simultaneously by minimizing (37) during the decision steps of the DRL agent.

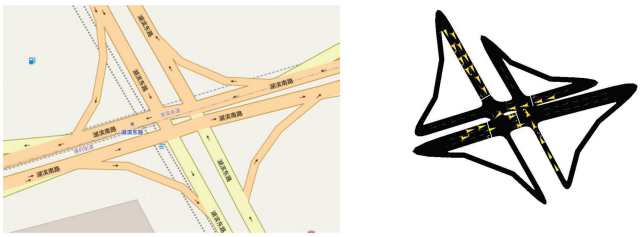
Algorithm 2 presents a pseudocode of GA-DRL training procedure. Also, since the computational complexity of GA-DRL mostly relies on the **Algorithm** 1, we neglect the analysis of computational complexity of **Algorithm** 2 without loss of generality.

V. Performance Evaluation

In this section, we first provide parameter settings for simulations. We then study the convergence of GA-DRL. Finally, we compare the performance of GA-DRL with four DAG task scheduling benchmarks in terms of the overall task completion time.

Algorithm 2: Training Process for GA-DRL

Input: GAT structure, DDQN structure, learning rate μ, and decision step $\mathcal{K}$ for copying θ^{t} from θ^{p}
Output: GNN with parameters $\mathcal{W}$ and $\mathcal{A}$, DDQN with parameters θ^{p} and θ^{t}
for each training episode **do**
  Consider a DAG task $\mathcal{G} = (\mathcal{B}, \mathcal{E})$ and a VC $\mathcal{V}$
  Obtain list $\mathcal{L}^{rank}$ using (19)-(23)
  for decision step $k = 1 \cdots K$ **do**
    For current subtask $b_{\tau(k)}$ use Algorithm 1 to extract its feature $h^{(L)}_{\tau(k)}$.
    The predict Q-network takes $s^{(k)} = \left\{h^{(L)}_{\tau(k)}, \mathcal{I}^{(k-1)}, \mathcal{A}^{(k)}, \mathcal{O}^{(k)}\right\}$ and performs an action $a^{(k)}$ according to ϵ-greedy policy
    Next decision step $s^{(k+1)}$ is obtained, and DRL agent receives a reward $r^{(k)}$ in (28)
    Store $\{s^{(k)}, a^{(k)}, r^{(k)}, s^{(k+1)}\}$ in the buffer $\mathcal{R}$
    Sample mini-batch $\mathcal{D}$ from $\mathcal{R}$ uniformly at random
    Use $\mathcal{D}$ to train GAT with parameters $\mathcal{W}$, $\mathcal{A}$ and DDQN with θ^{p} and θ^{t} jointly by minimizing (37)
    Update GAT and predict Q-network every step
    Set target Q-network to be a copy of the predict Q-network every $\mathcal{K}$ decision steps
  end for
end for



(a) Real-world traffic region. (b) Vehicle trajectory.

Fig. 8. VC network visualization.

A. Simulation Setting

Simulation environment: All neural networks considered in this work are implemented using PyTorch 2.0.0 [44] and Python 3.8.1 platforms, and Adam [45] is leveraged to optimize networks. In our simulations, we consider a real-world highway traffic region as shown in Fig. 8(a) of size 1km $\times$ 1km in Xiamen, China, obtained from OpenStreetMap [10]. Moreover, SUMO [11] is utilized to import mobile vehicles using the mobility model developed in (1)-(2), and subsequently emulate a real-world VC as shown in Fig. 8(b).

Parameter setting of vehicles: We assume that the computation capability of each vehicle is uniformly distributed in [1, 10] (in GHz) [19], and the distance between different vehicles during the task scheduling process are captured by SUMO. Also, we have $\beta = -17.8$ (indB) [29], fixed transmit power $P_m = 0.1$ (inW), $\forall v_m \in \mathcal{V}$, channel bandwidth $W = 10$ (in MHz), and noise power $\sigma^2 = 10^{-13}$ (inW). The arrival time of each vehicle, i.e., AT_m, is assumed to be uniformly distributed in [1, 5] (in second) for analytical simplicity, and $\mu_g = 50$ (in Kilometres per hour) with $\sigma_g = 10$.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.

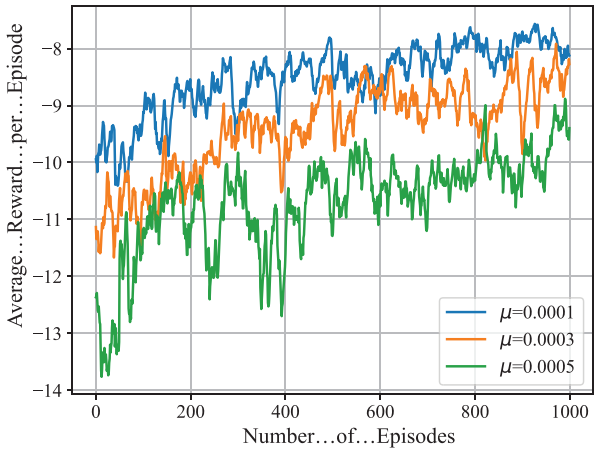


Fig. 9. Average reward under different learning rates.

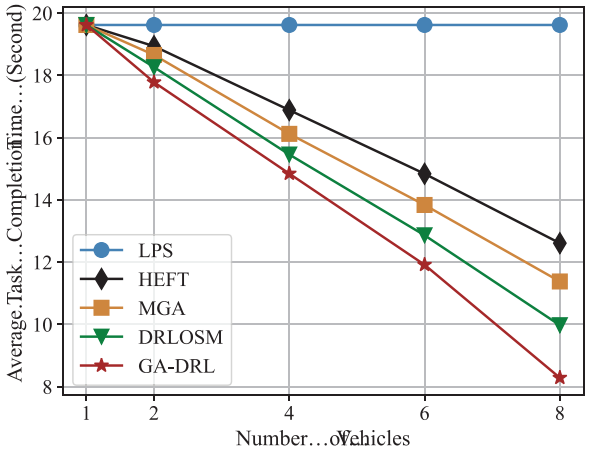


Fig. 10. Performance evaluations upon considering the different numbers of vehicles within VC.

Parameter setting of DAG tasks: The task owner has a DAG task which is generated according to [5]. The computation workload of each subtask is uniformly distributed in [1, 2] (in Gigaclock cycles) [26] and the transmission data size of each edge is uniformly distributed in [100, 500] (in KB) [26].

Parameter setting of neural networks: In our experiments, we found that our GAT could achieve a high performance with $L = 2$, $Z = 4$, where $W^{(1)} \in \mathbb{R}^{4\times 16}$, $A^{(1)} \in \mathbb{R}^{32\times 1}$, and $W^{(2)} \in \mathbb{R}^{16\times 32}$, $A^{(2)} \in \mathbb{R}^{64\times 1}$. Also, in DDQN, we used deep neural networks with two hidden layers with 256 dimensions, initial ϵ-greedy policy and discount factor are set to $\epsilon = 0.9$ and $\gamma_2 = 0.9$, respectively.

B. Convergence Performance

In Fig. 9, we depict the convergence behavior of GA-DRL with respect to the number of episodes. This experiment was conducted considering 20 subtasks, 5 layers, and 10 vehicles. Note that the best convergence and reward values are achieved when the GA-DRL's learning rate is 0.0001. We thus use this setting in the presentation of the rest of the results. On the other hand, as the learning rate increases from 0.0001 to 0.0005, the average reward is significantly decreased due to the instability of learning. As a result, we fix the learning rate of the GA-DRL to 0.0001 when comparing it with benchmarks in the following.

C. Benchmarks

To study the performance of GA-DRL, we implement four DAG task scheduling benchmarks, including LPS, HEFT [12], MGA [15], and DRLOSM [46] as detailed below.

- *Local Processing Scheme (LPS):* All subtasks are processed locally by the task owner itself without offloading to other vehicles.
- *Heterogeneous Earliest Finish Time (HEFT) [12]:* All subtasks are first sorted according to their ranking value in (25). The subtasks are assigned to the vehicles that can complete them in the shortest time. The HEFT algorithm does not take into account the constraint of V2V transmission (C5) since it was designed for a static computing environment. We assume that subtasks-to-vehicles allocations that do not satisfy constraint (C5) are executed locally.
- *Modified Genetic Algorithm (MGA) [15]:* MGA considers an integer encoding to denote subtask-to-vehicle assignments. The assignments with high fitness (i.e., low task completion time) are stochastically selected to perform crossover (i.e., exchange their processing vehicles). Finally, a mutation (i.e., changing the processing vehicle) is adopted to avoid early convergence. MGA considers a VC environment satisfying V2V communication constraint (C5).
- *DRL Offloading Scheduling Method (DRLOSM) [46]:* DRLOSM is an improved version of the method proposed in [46]. All subtasks are first sorted according to their ranking value in (25). DRLOSM uses a DDQN architecture, where at decision step k, the raw feature of *current subtask* $b_{\tau(k)}$ is integrated in $s^{(k)}$ without the use of GNNs. DRLOSM also satisfies the V2V communication constraint (C5) through an action mask module.

D. Simulation Results of Randomly Generated DAG Tasks

We conduct performance evaluations by analyzing the average completion time of DAG tasks for various numbers of layers[7] of DAG tasks, subtasks, and vehicles in the network. The results are the average performance obtained via 100 independent Monte-Carlo iterations. Also, to compare the generalizability of DRLOSM and our GA-DRL, during the training period, we use the same DAG task topology, while deploying them for various DAG task topologies under performance evaluation.

1) Impact of the Number of Vehicles in VC: The results depicted in Fig. 10 demonstrate the effect of incrementally increasing the number of vehicles from 1 to 8 on the overall completion time for the DAG task. This experiment was

[7]The number of layers of a DAG task refers to the length of the longest path from the starting subtask to the finishing subtask. For a DAG task with a fixed number of subtasks, as the number of layers increases/decreases, there are more/less subtasks that are successors of the same subtask, implying a higher/lower potential for parallelism during the task execution.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.

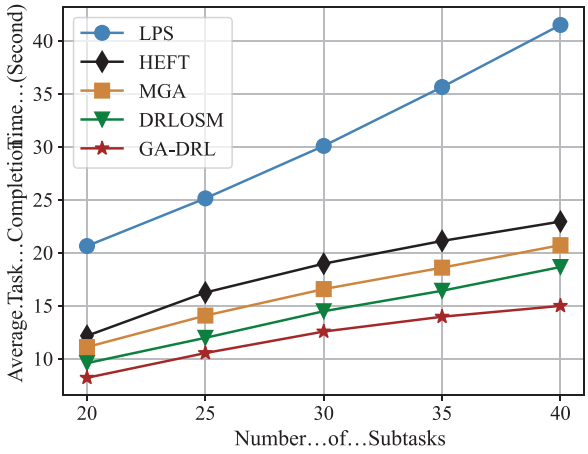


Fig. 11. Performance evaluations upon considering the different numbers of subtasks within DAG task.

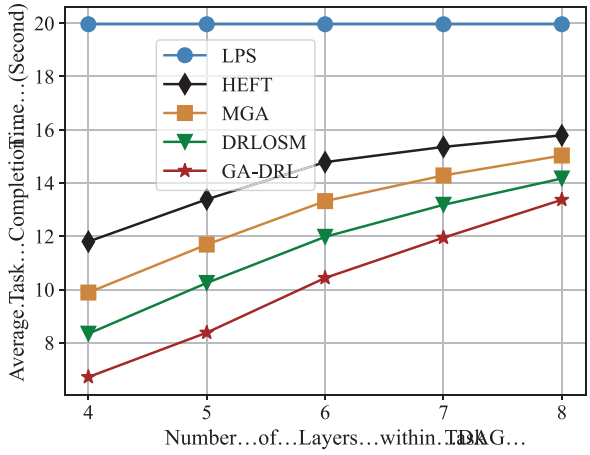


Fig. 12. Performance evaluations upon considering the different numbers of layers within DAG task.

designed with a total of 20 subtasks, evenly distributed across 5 layers. Given that a maximum of 4 subtasks per layer can be processed concurrently in parallel, the experiment capped the number of vehicles at 8. This limit accounts for the reality that although only 4 vehicles can be simultaneously deployed to process the subtasks, not all vehicles may be available for task processing due to their differing mobility patterns and V2V connections. We can observe that when only one vehicle is involved in VC, all DAG subtasks have to be executed sequentially and locally, resulting in the same completion time across different algorithms. However, as the number of vehicles increases, the completion of DAG tasks is significantly accelerated due to the sufficient computation resources. Comparing with DRLOSM which utilizes handcrafted features to represent subtasks, our GA-DRL can extract more topological information from DAG task by leveraging GAT and thus achieves a lower task completion time. Overall, GA-DRL outperforms other algorithms in terms of task completion time. It is 9.38% better than LPS, 6.12% better than HEFT, 4.69% better than MGA, 2.65% better than DRLOSM at 2 vehicles; and is 57.76% better than LPS, 34.28% better than HEFT, 27.18% better than MGA, and 16.93% better than DRLOSM at 8 vehicles.

2) Impact of the Number of Subtasks: In Fig. 11, we can see the evaluation of the completion time for DAG tasks as the number of subtasks increases.[8] In this experiment, we set the number of vehicles involved in the VC at 12, and the number of layers at 5. The results show that our proposed GA-DRL algorithm outperforms the other four benchmarks, achieving faster task completion times. Additionally, Fig. 11 demonstrates the effectiveness of GA-DRL compared to conventional DRL in terms of generalizability, as an increase in the number of subtasks, the topologies of DAG tasks become more complicated, making the DRLOSM algorithm, which relies solely on handcrafted features without the usage of GNNs, unable to capture the topological information of the newly generated DAG task topologies. On the other hand, our GA-DRL algorithm benefits from more informative subtasks' features, which are automatically learned from GAT, making its models well generalizable to unseen DAG task topologies. In summary, the performance of GA-DRL in terms of the task completion time is 60.29% better than LPS, 32.59% better than HEFT, 26.14% better than MGA, 14.15% better than DRLOSM at 20 subtasks; and is 63.91% better than LPS, 34.37% better than HEFT, 27.67% better than MGA, and 19.67% better than DRLOSM at 40 subtasks.

3) Impact of the Number of Layers Within DAG Task: In Fig. 12, it is evident that changing the number of DAG task layers from 4 to 8 has a notable impact on the completion time of the DAG task. In this result, we considered 20 subtasks and 10 vehicles. It can be observed that increasing the number of layers leads to a longer completion time. This is because, as the number of layers increases, the parallelism of the DAG task decreases, resulting in more subtasks being executed in a sequential manner. This, in turn, leads to a longer task completion time. Consequently, the trend of curves in Fig. 12 is in contrast to the trend observed in Fig. 11 (i.e., the performance gap between GA-DRL and baseline methods decreases along the x-axis). The performance of GA-DRL in terms of the task completion time is 66.38% better than LPS, 43.11% better than HEFT, 32.11% better than MGA, 19.52% better than DRLOSM at 4 layers; and is 33.05% better than LPS, 15.37% better than HEFT, 11.06% better than MGA, and 5.26% better than DRLOSM at 8 layers.

E. Simulation Results for Real Application DAG Task

In Fig. 13, we illustrate a real-world DAG task of a modified molecular dynamic code. The topology of the task entailing the number of subtasks and edges are inspired by [12], while the computation workload and transmission data size of different subtasks are set according to the parameter settings of the aforementioned DAG tasks. Also, 12 vehicles are considered in the system. Table III presents the performance comparison of various benchmarks, except LPS,[9] with respect to the

[8]Given the fixed number of layers within DAG task, the added subtasks are uniformly distributed among different layers.

[9]LPS is excluded, since it takes no algorithm running time, while we have demonstrated above that the task completion time of LPS is always worse than that of other benchmarks.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.

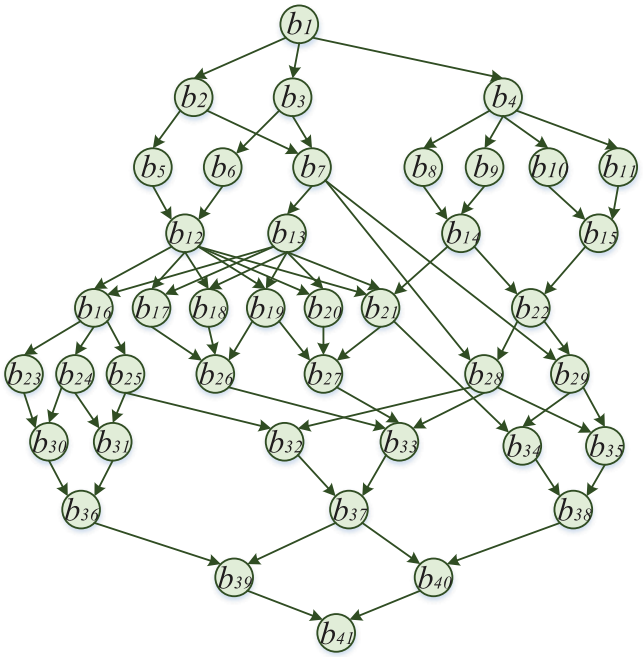


Fig. 13. The DAG task model of the molecular dynamics code [12].

DAG task completion time (in seconds) and the algorithm running time (in seconds). As can be seen, MGA has the longest running time among the benchmarks due to its internal iteration time for convergence. Although the algorithm running time of GA-DRL is higher compared to DRLOSM, the overall completion time (the sum of algorithm running time and task completion time) of GA-DRL and DRLOSM are 19.31 and 21.97 (in seconds), respectively. This indicates that GA-DRL outperform DRLOSM in terms of time-efficiency. The performance of GA-DRL is 31.72% better than HEFT, 26.18% better than MGA, and 16.93% better than DRLOSM. In summary, simulation results verify that our proposed GA-DRL algorithm offers an efficient and commendable reference in scheduling DAG tasks over dynamic VCs.

VI. Conclusion and Future Work

In this paper, we focused on scheduling DAG tasks using a combination of GNNs and DRL. We approached the problem by modeling it as an MDP, and using a GAT module to extract features for each subtask in the DAG task topology. We then integrated the GAT with a DDQN to allocate subtasks to vehicles while taking into account the dynamics and heterogeneity of the vehicles. Our GAT uses multiple heads to enhance information for important subtasks and aggregates topological information from both subtasks' predecessors and successors. We also incorporated a non-uniform neighborhood sampling methodology to improve the GAT's generalizability. Our evaluations showed that our GA-DRL method outperforms benchmarks in terms of task completion time. Our proposed GA-DRL framework holds potential for application in a range of other scenarios. One such example is the scheduling of DAG tasks over cloud servers with dynamic availabilities, which may experience unpredictable downtimes. The exploration of these applications is an exciting avenue for future work. Future research could also delve into the potential of integrating other emerging DRL methodologies, such as meta-reinforcement learning, into GNNs to improve the task scheduling performance for both static and dynamic cloud networks.

References

[1] Z. Lai, Y. C. Hu, Y. Cui, L. Sun, N. Dai, and H.-S. Lee, "Furion: Engineering high-quality immersive virtual reality on today's mobile devices," *IEEE Trans. Mobile Comput.*, vol. 19, no. 7, pp. 1586–1602, Jul. 2020.

[2] T. Taleb, A. Ksentini, M. Chen, and R. Jantti, "Coping with emerging mobile social media applications through dynamic service function chaining," *IEEE Trans. Wireless Commun.*, vol. 15, no. 4, pp. 2859–2871, Apr. 2016.

[3] L. F. Bittencourt, R. Sakellariou, and E. R. M. Madeira, "DAG scheduling using a lookahead variant of the heterogeneous earliest finish time algorithm," in *Proc. Eur. Conf. Parallel Process.*, 2010, pp. 27–34.

[4] G. C. Sih and E. A. Lee, "A compile-time scheduling heuristic for interconnection-constrained heterogeneous processor architectures," *IEEE Trans. Parallel Distrib. Syst.*, vol. 4, no. 2, pp. 175–187, Feb. 1993.

[5] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 3, pp. 682–694, Mar. 2014.

[6] H. Kanemitsu, M. Hanada, and H. Nakazato, "Clustering-based task scheduling in a large number of heterogeneous processors," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 11, pp. 3144–3157, Nov. 2016.

[7] Z. Liu, M. Liwang, S. Hosseinalipour, H. Dai, Z. Gao, and L. Huang, "RFID: Towards low latency and reliable DAG task scheduling over dynamic vehicular clouds," *IEEE Trans. Veh. Technol.*, vol. 72, no. 9, pp. 12139–12153, Sep. 2023.

[8] M. Barbera, S. Kosta, A. Mei, and J. Stefa, "To offload or not to offload? The bandwidth and energy costs of mobile cloud computing," in *Proc. IEEE Conf. Comput. Commun.*, 2013, pp. 1285–1293.

[9] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, Oct. 2016.

[10] M. Haklay and P. Weber, "OpenStreetMap: User-generated street maps," *IEEE Pervasive Comput.*, vol. 7, no. 4, pp. 12–18, Oct. 2008.

[11] P. A. Lopez et al., "Microscopic traffic simulation using SUMO," in *Proc. Int. Conf. Intell. Transp. Syst. (ITSC)*, 2018, pp. 2575–2582.

[12] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.

[13] Y. Sahni, J. Cao, L. Yang, and Y. Ji, "Multihop offloading of multiple DAG tasks in collaborative edge computing," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4893–4905, Mar. 2021.

[14] Q. Shen, B.-J. Hu, and E. Xia, "Dependency-aware task offloading and service caching in vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 12, pp. 13182–13197, Dec. 2022.

[15] F. Sun et al., "Cooperative task scheduling for computation offloading in vehicular cloud," *IEEE Trans. Veh. Technol.*, vol. 67, no. 11, pp. 11049–11061, Nov. 2018.

[16] H. Liu, H. Zhao, L. Geng, and W. Feng, "A policy gradient based offloading scheme with dependency guarantees for vehicular networks," in *Proc. IEEE Glob. Commun. Conf. (GLOBECOM)*, 2020, pp. 1–6.

[17] Y. Liu et al., "Dependency-aware task scheduling in vehicular edge computing," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 4961–4971, Jun. 2020.

[18] J. Shi, J. Du, J. Wang, J. Wang, and J. Yuan, "Priority-aware task offloading in vehicular fog computing based on deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 16067–16081, Dec. 2020.

[19] J. Xie et al., "Advanced dropout: A model-free methodology for Bayesian dropout optimization," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 9, pp. 4605–4625, Sep. 2022.

[20] J. Yan, S. Bi, and Y. J. A. Zhang, "Offloading and resource allocation with general task graph in mobile edge computing: A deep reinforcement learning approach," *IEEE Trans. Wireless Commun.*, vol. 19, no. 8, pp. 5404–5419, Aug. 2020.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.

[21] M. S. Mekala et al., "A DRL-based service offloading approach using DAG for edge computational orchestration," *IEEE Trans. Comput. Social Syst.*, early access, Apr. 12, 2022, doi: 10.1109/TCSS.2022.3161627.
[22] J. Wang, J. Hu, G. Min, A. Y. Zomaya, and N. Georgalas, "Fast adaptive task offloading in edge computing based on meta reinforcement learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 1, pp. 242–253, Jan. 2021.
[23] M. Goudarzi, M. S. Palaniswami, and R. Buyya, "A distributed deep reinforcement learning technique for application placement in edge and fog computing environments," *IEEE Trans. Mobile Comput.*, vol. 22, no. 5, pp. 2491–2505, May 2023.
[24] Z. Hu, J. Tu, and B. Li, "Spear: Optimized dependency-aware task scheduling with deep reinforcement learning," in *Proc. 39th IEEE Int. Conf. Distrib. Comput. Syst. (ICDCS)*, 2019, pp. 2037–2046.
[25] X. Wei, L. Cai, N. Wei, P. Zou, J. Zhang, and S. Subramaniam, "Joint UAV trajectory planning, DAG task scheduling, and service function deployment based on DRL in UAV-empowered edge computing," *IEEE Internet Things J.*, vol. 10, no. 14, pp. 12826–12838, Jul. 2023.
[26] L. Geng, H. Zhao, J. Wang, A. Kaushik, S. Yuan, and W. Feng, "Deep reinforcement learning based distributed computation offloading in vehicular edge computing networks," *IEEE Internet Things J.*, vol. 10, no. 14, pp. 12416–12433, Jul. 2023.
[27] F. Basic, A. Aral and I. Brandic, "Fuzzy handoff control in edge offloading," in *Proc. IEEE Int. Conf. Fog Comput. (ICFC)*, 2019, pp. 87–96.
[28] M. Taneja and A. Davy, "Resource aware placement of IoT application modules in fog-cloud computing paradigm," in *Proc. IFIP/IEEE Symp. Integr. Netw. Serv. Manag. (IM)*, 2017, pp. 1222–1228.
[29] Y. Sun et al., "Adaptive learning-based task offloading for vehicular edge computing systems," *IEEE Trans. Veh. Technol.*, vol. 68, no. 4, pp. 3061–3074, Apr. 2019.
[30] Z. Ning, P. Dong, X. Kong, and F. Xia, "A cooperative partial computation offloading scheme for mobile edge computing enabled Internet of Thing," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4804–4814, Jun. 2019.
[31] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018, pp. 1–12.
[32] S. Yousefi, E. Altman, R. El-Azouzi, and M. Fathy, "Analytical model for connectivity in vehicular ad hoc networks," *IEEE Trans. Veh. Technol.*, vol. 57, no. 6, pp. 3341–3356, Nov. 2008.
[33] S. Misra and S. Bera, "Soft-VAN: Mobility-aware task offloading in software-defined vehicular network," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 2071–2078, Feb. 2020
[34] Z. He, L. Wang, H. Ye, G. Y. Li, and B.-H. F. Juang, "Resource allocation based on graph neural networks in vehicular communications," in *Proc. IEEE Glob. Commun. Conf. (GLOBECOM)*, 2020, pp. 1–5.
[35] Y. Li, J. Li, Z. Lv, H. Li, Y. Wang, and Z. Xu, "GASTO: A fast adaptive graph learning framework for edge computing empowered task offloading," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 932–944, Jun. 2023.
[36] H. Lee, S. Cho, Y. Jang, J. Lee, and H. Woo, "A global DAG task scheduler using deep reinforcement learning and graph convolution network," *IEEE Access*, vol. 9, pp. 158548–158561, 2021.
[37] J. Chen, Y. Yang, C. Wang, H. Zhang, C. Qiu, and X. Wang, "Multi-task offloading strategy optimization based on directed acyclic graphs for edge computing," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9367–9378, Jun. 2022.
[38] M. Liwang, Z. Gao, S. Hosseinalipour, Y. Su, X. Wang, and H. Dai, "Graph-represented computation-intensive task scheduling over air-ground integrated vehicular networks," *IEEE Trans. Services Comput.*, vol. 16, no. 5, pp. 3397–3411, Sep./Oct. 2023.
[39] J. Wang, C. Jiang, K. Zhang, T. Q. S. Quek, Y. Ren, and L. Hanzo, "Vehicular sensing networks in a smart city: principles, technologies and applications," *IEEE Wireless Commun.*, vol. 25, no. 1, pp. 122–132, Feb. 2018.
[40] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 229–256, May 1992.
[41] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.
[42] S. Wang, M. Lee, S. Hosseinalipour, R. Morabito, M. Chiang, and C. G. Brinton, "Device sampling for heterogeneous federated learning: Theory, algorithms, and implementation," in *Proc. IEEE Int. Conf. Comput. Commun. (INFOCOM)*, 2021, pp. 1–10.
[43] H. Ye, J. Wang, and Z. Li, "MIP reformulation for max-min problems in two-stage robust SCUC," *IEEE Trans. Power Syst.*, vol. 32, no. 2, pp. 1237–1247, Mar. 2017.
[44] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 8024–8035.
[45] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015, pp. 1–15.
[46] W. Zhan et al., "Deep-reinforcement-learning-based offloading scheduling for vehicular edge computing," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5449–5465, Jun. 2020.
[47] X. Yang, H. Luo, Y. Sun, and M. Guizani, "A novel hybrid-ARPPO algorithm for dynamic computation offloading in edge computing," *IEEE Internet Things J.*, vol. 9, no. 23, pp. 24065–24078, Dec. 2022.
[48] D. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and accurate deep network learning by exponential linear units (ELUs)," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016, pp. 1–14.
[49] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
[50] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.
[51] L. Hou, J. Yan, C. Wang, and L. Ge, "A simultaneous multi-round auction design for scheduling multiple charges of battery electric vehicles on highways," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 8024–8036, Jul. 2022.
[52] S. Guo, J. Liu, Y. Yang, B. Xiao, and Z. Li, "Energy-efficient dynamic computation offloading and cooperative task scheduling in mobile cloud computing," *IEEE Trans. Mobile Comput.*, vol. 18, no. 2, pp. 319–333, Feb. 2019.
[53] Y. Wu, W. Zhang, N. Guan, and Y. Ma, "TDTA: Topology-based real-time DAG task allocation on identical multiprocessor platforms," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 11, pp. 2895–2909, Nov. 2023.



Zhang Liu (Graduate Student Member, IEEE) received the B.S. degree in communication engineering from the East China University of Science and Technology, Shanghai, China, in 2019. He is currently pursuing the Ph.D. degree with the School of Informatics, Xiamen University, Xiamen, China.

His research interests include wireless communication systems, graph theory, and computation offloading as well as resource allocation in vehicular networks.



Lianfen Huang (Member, IEEE) received the B.S. degree in radio physics and the Ph.D. degree in communication engineering from Xiamen University, Xiamen, China, in 1984 and 2008, respectively.

She was a Visiting Scholar with Tsinghua University, Beijing, China, in 1997. She is a Professor with the Department of Communication Engineering, Xiamen University. Her current research interests include wireless communication, wireless network, and signal processing.



Zhibin Gao (Member, IEEE) received the B.S. degree in communication engineering, the M.S. degree in radio physics, and the Ph.D. degree in communication engineering from Xiamen University, Xiamen, China, in 2003, 2006, and 2011, respectively.

He is a Professor with Navigation Institute, Jimei University, Xiamen, China, where he worked as a Senior Engineer of communication engineering. His research interests include Internet of Vehicles, wireless network resource management, and signal processing.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.



Manman Luo received the B.S. degree in electronic information science and technology from Nanjing Agricultural University in 2020. She is currently pursuing the master's degree with the Department of Electronic Engineering, School of Electronic Science and Technology, Xiamen University.

Her current research interests include computation offloading, task scheduling, and deep reinforcement learning.



Seyyedali Hosseinalipour (Member, IEEE) received the Ph.D. degree in electrical engineering from North Carolina State University in 2020.

He is currently an Assistant Professor with the Department of Electrical Engineering, University at Buffalo SUNY, Buffalo, NY, USA. His research interests include 6G, machine learning, federated learning, fog and edge computing, and network optimization.



Huaiyu Dai (Fellow, IEEE) received the B.E. and M.S. degrees in electrical engineering from Tsinghua University, Beijing, China, in 1996 and 1998, respectively, and the Ph.D. degree in electrical engineering from Princeton University, Princeton, NJ, in 2002.

He was with Bell Labs, Lucent Technologies, Holmdel, NJ, in 2000, and with AT&T Labs-Research, Middletown, NJ, in 2001. He is currently a Professor of electrical and computer engineering with North Carolina State University, Raleigh. His research interests are in the general areas of communication systems and networks, advanced signal processing for digital communications, and communication theory and information theory. His current research focuses on networked information processing and crosslayer design in wireless networks, cognitive radio networks, network security, and associated information-theoretic and computation-theoretic analysis. He was a co-recipient of best paper awards at 2010 IEEE International Conference on Mobile Ad-hoc and Sensor Systems, 2016 IEEE INFOCOM BIGSECURITY Workshop, and 2017 IEEE International Conference on Communications. He has served as an Editor for IEEE TRANSACTIONS ON COMMUNICATIONS, IEEE TRANSACTIONS ON SIGNAL PROCESSING, and IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS. He is currently an Area Editor in charge of wireless communications for IEEE TRANSACTIONS ON COMMUNICATIONS. He co-edited two special issues of EURASIP journals on distributed signal processing techniques for wireless sensor networks, and on multiuser information theory and related applications, respectively. He co-chaired the Signal Processing for Communications Symposium of IEEE Globecom 2013, the Communications Theory Symposium of IEEE ICC 2014, and the Wireless Communications Symposium of IEEE Globecom 2014.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:09 UTC from IEEE Xplore. Restrictions apply.